Frequency Estimation of Strict-Turnstile Data Stream with L_1 α-Bounded Deletion Property

WONG, Hok Fong Silas (1155189917)

September 19, 2026

Abstract

Frequency estimation is a fundamental problem in data stream processing with applications ranging from network monitoring to database analytics. While classical algorithms like Misra-Gries and Space-Saving were designed for insertion-only streams, and Count-Min Sketch and Count-Sketch inherently support the turnstile model, real-world applications often involve both insertions and deletions under additional constraints. This project investigates extensions of these four algorithms to the strict turnstile model with the L_1 α-bounded deletion property, which guarantees that the total update volume is at most α times the final L_1 norm. We propose modifications to correctly handle signed updates while preserving core guarantees, introducing symmetric deletion rules for counter-based algorithms and sampling-based optimizations for sketches that reduce space from logarithmic to $\mathcal{O}(\log \alpha)$ dependence. Extensive experiments evaluate performance across varying α values, stream lengths, and data distributions, demonstrating that the α-property significantly tightens error bounds and reduces space requirements. Motivated by the observed trade-offs-where counter-based methods achieve high accuracy but scale linearly with α, while sketches are space-efficient but less accurate for heavy hitters. We propose an enhanced conceptual design: a *Dyadic Count-Min Hierarchy with an Adaptive Counter Cache*. This hybrid structure combines a tree of sampled Count-Min sketches (space logarithmic in α) with a small deterministic cache that preserves the accuracy of counter-based methods for the heaviest items. The design offers a promising direction for achieving near-optimal space-accuracy trade-offs in future streaming systems.

1 Introduction

1.1 Background

Data streams have become ubiquitous in modern data-intensive applications, from network traffic monitoring and financial transaction analysis to database synchronization and e-commerce analytics [Mut05]. In these settings, massive volumes of data arrive continuously, often at high velocity, and systems must extract meaningful insights under strict constraints on memory and processing time. Streaming algorithms address this challenge by processing data in a single pass while maintaining compact summaries, or sketches, that support approximate queries with strong theoretical guarantees.

Among the most fundamental problems in data stream processing is frequency estimation. For a universe of items, we wish to approximate the number of times each item has appeared. One closely related research question is on the frequent items (or heavy hitters) problem, which asks to identify all items whose frequencies exceed a specified fraction of the total stream length. These problems underpin numerous applications, including anomaly detection in networks [WP05], click-stream analysis [GÖ03][MAEA05], and quantile approximation [CM05][WLYC13].

Over the past two decades, a rich literature has developed around frequency estimation. Classical algorithms fall into two main categories [ZAEA+25]. Counter-based algorithms, such as Misra-Gries [MG82] and Space-Saving [MAEA05], maintain a fixed set of item-count pairs and achieve deterministic guarantees with space $\mathcal{O}(1/\epsilon)$. Sketch-based algorithms, including Count-Min [CM05] and Count-Sketch [CCFC02], use randomized hash functions to provide probabilistic error bounds with space $\mathcal{O}(\frac{1}{\epsilon}\log(\frac{1}{\delta}))$. Both families were originally designed for the insertion-only model, where all updates are positive.

Many real-world streams, however, involve both insertions and deletions. Network flows may expire, purchases may be returned, or database records may be updated. To capture such dynamics, the turnstile model allows updates of either sign, but this generality comes at a cost: known lower bounds show that solving frequency estimation in the turnstile model requires a multiplicative

$\Theta(\log n)$ factor more space than in the insertion-only model [JST11][KNW10][BDW18]. This gap arises because adversarial streams can insert a large number of items and then delete nearly all of them, leaving a tiny final signal that is hard to distinguish from noise.

In practice, many turnstile streams are not adversarial: deletions do not happen arbitrarily, and real streams often exhibit a bounded relationship between insertions and deletions. Motivated by this observation, Jayaram and Woodruff [JW13] introduced the α-bounded deletion model (also called the α-property), which parameterizes the degree of churn in the stream.

In this project, we also specifically consider the strict turnstile model, which requires that the frequency of every item remains non-negative throughout the stream [Mut05]. While this may seem restrictive, it reflects a fundamental property of many real-world systems, as deletions are almost always coupled with prior insertions. Moreover, in systems that track counts or inventories, negative frequencies would correspond to impossible states (e.g., a negative number of active orders, or a negative packet count in a network flow). Therefore, the strict turnstile condition is not an artificial mathematical constraint but rather a faithful reflection of how insertions and deletions interact in practice. Combined with the α-bounded property, which limits the relative volume of deletions, this model captures a wide range of realistic streaming workloads while enabling stronger algorithmic guarantees than the fully adversarial turnstile model.

1.2 The Strict-Turnstile L_1 α-Bounded Model

We begin by formalizing the stream model. Let $\mathcal{U}$ be a universe of items and $\mathcal{S}$ be a data stream consist of a sequence of updates (e_t, Δ_t) for $t = 1, \ldots, m$, where $e_t \in \mathcal{U}$ and $\Delta_t \in \mathbb{Z}$.

For each item $e \in \mathcal{U}$, define the *total inserted weight* I_e and the *total deleted weight* D_e as:

$$I_e = \sum_{\substack{t: e_t = e \\ \Delta_t > 0}} \Delta_t, \qquad D_e = \sum_{\substack{t: e_t = e \\ \Delta_t < 0}} |\Delta_t|.$$

Then the final *frequency vector* $\mathbf{f} \in \mathbb{R}^{|\mathcal{U}|}$ is $\mathbf{f} := \mathbf{I} - \mathbf{D}$, where $\mathbf{I} = (I_e)_{e \in \mathcal{U}}$ and $\mathbf{D} = (D_e)_{e \in \mathcal{U}}$.

Definition 1 (Strict-Turnstile Streams)**.** *For any time t, let $f_e^{(t)}$ denote the* cumulative frequency *of item e after processing the first t updates, i.e.*

$$f_e^{(t)} = \sum_{s \le t} \Delta_s \cdot \mathbb{1}[e_s = e],$$

where $\mathbb{1}$[condition] is equal to 1 if condition *is true, and 0 otherwise.*

A stream is said to be in the ***strict-turnstile model*** *if for every item $e \in \mathcal{U}$ and at every time $t \in \{1, \ldots, m\}$, the cumulative frequency $f_e^{(t)}$ never becomes negative.*

Definition 2 (L_1 α-Bounded Property)**.** *A stream satisfies the L_1 α-**bounded deletion property** for some $\alpha \ge 1$ if*

$$\|\mathbf{I} + \mathbf{D}\|_1 \le \alpha \|\mathbf{f}\|_1,$$

where $\|x\|_1 = \sum_e |x_e|$. Equivalently, the total absolute weight of all updates is at most α times the final L_1 norm.

In the *strict-turnstile L_1 α-bounded stream setting*, where frequencies never become negative throughout the stream, this definition simplifies considerably. Since all frequencies remain non-negative, the final L_1 norm is simply $\|\mathbf{f}\|_1 = \sum_e f_e^{(m)}$, and the condition becomes:

$$\sum_{t=1}^{m} |\Delta_t| \le \alpha \sum_e f_e.$$

For the L_1 version, a stream satisfies the α-property if the total absolute weight of all updates is at most α times the final L_1 norm. In the strict turnstile setting where frequencies never become negative, this is equivalent to requiring that the number of deletions $\|\mathbf{D}\|_1$ and insertions $\|\mathbf{I}\|_1$ satisfy $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 \le \alpha(\|\mathbf{I}\|_1 - \|\mathbf{D}\|_1)$, i.e., $\|\mathbf{D}\|_1 \le \|\mathbf{I}\|_1 \cdot (\alpha - 1)/(\alpha + 1)$. Thus the deletion fraction is bounded by a known constant α. This model interpolates smoothly between insertion-only streams ($\alpha = 1$) and full turnstile streams ($\alpha = \text{poly}(n)$), and captures many realistic workloads such as sliding windows over network traffic, and inventory systems with returns, which will be described in the next section.

1.3 Real-World Examples

We now present two concrete scenarios from diverse scenarios, arise from network monitoring and e-commerce analytics, where both the α-bounded property and strict-turnstile property are both satisfied.

Sliding Window over Network Traffic. Sliding window queries are fundamental in network monitoring (e.g., Cisco's NetFlow [DCC+11], intrusion detection systems). While the total history can be arbitrarily long, the window size limits the amount of "active" data. Consider a router that monitors packets sent between each source-destination pair over a sliding window of the last W seconds. As new packets arrive, they are inserted; when a packet's age exceeds W, it is deleted.

Each packet arrival at time t for flow (src, dst) generates an insertion $((src, dst), +1)$. Exactly W seconds later, the same packet generates a deletion $((src, dst), -1)$. Thus the stream consists of matched insertions and deletions, delayed by the window size. The deletion only occurs after the corresponding insertion, so the frequency of each flow is always non-negative. At any moment, the frequency equals the number of packets in the last W seconds.

Let $\|\mathbf{I}\|_1$ be the total number of insertions over the whole stream lifetime, and $\|\mathbf{D}\|_1$ be the total deletions. For a sliding window, every insertion is eventually deleted (except possibly at the very end if we stop before the window expires), so $\|\mathbf{I}\|_1 \approx \|\mathbf{D}\|_1$ for long streams. The final L_1 norm $\|\mathbf{f}\|_1$ is the number of packets still in the window at the end. If the average packet rate is r packets per second, then over a long observation period T we have $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 \approx 2rT$, while $\|\mathbf{f}\|_1 \leq rW$. Hence, for a fixed query time T, we can safely assume that $\alpha = \mathcal{O}(\lceil 2T/W \rceil)$. A great consequence is that α does not grow with the total number of packets processed, but it depends only on the ratio T/W.

E-Commerce Inventory with Returns. An online retailer tracks the net number of successful orders for each product to manage product supply and reordering decisions [ZAEA+25]. When a customer places an order, the product count increases; if the order is later returned or canceled, the count decreases. In a well-managed business, orders are fulfilled after some time, so deletions always correspond to previously inserted orders.

Each update is a pair $(\text{product}_{id}, \Delta_t)$, where $\Delta_t > 0$ indicates the order placed, and $\Delta_t < 0$ indicates an order returned or canceled. An order can only be fulfilled after it has been placed, so we never delete an item that hasn't been inserted. The frequency of each product (active orders) is always non-negative.

Let $\|\mathbf{I}\|_1$ be the total number of orders placed over a long period, and $\|\mathbf{D}\|_1$ be the total number of units returned or canceled. For a stable business, the number of active orders at any time is roughly constant, L. Over a long period, $\|\mathbf{I}\|_1 \approx \|\mathbf{D}\|_1$ (each order eventually leaves the system). The total updates $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1$, and the final L_1 norm $\|\mathbf{f}\|_1 = L$ is the total net number of successful orders at the end of the period, which is then needed for inventory planning.

In typical e-commerce scenarios, α is a small constant, independent of the total number of transactions n. For example, a popular item might have 10,000 units ordered and 2,000 units returned over a month, with 8,000 net successful orders. Then $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 = 12,000$, $\|\mathbf{f}\|_1 = 8,000$, and $\alpha = 1.5$. If return rates are lower, say 200 returns out of 10,000 orders, then $\alpha = (10,000 + 200)/9,800 \approx 1.04$. Note that higher return rates yield larger α, meaning more churn relative to the final net count.

Classical turnstile algorithms store counters that must count up to the total transaction volume n, requiring $\mathcal{O}(\log n)$ bits per counter. However, as n can be in the millions and α is often small for a stable business (where $\alpha \in [1.04, 1.50]$ for most products), this yields substantial space savings if the space complexity of some frequency estimation algorithms on n can be partly or fully replaced by α.

1.4 Overview of the Report

Recent Work. Jayaram and Woodruff [JW18] demonstrated that for several fundamental problems, including heavy hitters, L_1 estimation, and inner product estimation, the $\log n$ factor in the turnstile lower bound can be replaced by $\log \alpha$ when the stream satisfies the α-property. Their work, however, focused primarily on Count-Sketch and left open the question of how counter-based methods like Misra-Gries and Space-Saving could be modified for improvement under bounded deletions. More recently, Zhao et al. [ZAEA+25] proposed SpaceSaving±, a deterministic extension of Space-Saving that achieves optimal space $\mathcal{O}(\beta/\epsilon)$ in a closely related bounded-deletion model where $\|\mathbf{D}\|_1 \leq (1 - 1/\beta)\|\mathbf{I}\|_1$. Their experimental results show that SpaceSaving± outperforms

existing sketches in accuracy across a range of distributions. Note that if a stream satisfies the α-property, then $\|\mathbf{D}\|_1 \leq \|\mathbf{I}\|_1 \cdot (\alpha - 1)/(\alpha + 1)$ and we can set $\beta = (\alpha + 1)/2$, yielding a bound of $\mathcal{O}(\beta/\epsilon) = \mathcal{O}(\frac{\alpha+1}{2\epsilon}) = \mathcal{O}(\alpha/\epsilon)$.

Despite these advances, a comprehensive study that systematically extends all four classic algorithms (Misra-Gries, Space-Saving, Count-Min, Count-Sketch) to the strict turnstile α-bounded deletion model is still lacking. Moreover, while the theoretical advantages of the α-property are well established, their practical implications of how different algorithms trade off space, accuracy, and update time under varying α and data skew, remain underexplored.

Our Contributions. This project addresses these gaps with the following work:

- We propose modifications to Misra-Gries and Space-Saving that correctly handle deletions while preserving the core invariants of the original algorithms. For each, we prove error bounds that scale with α, showing that with space $\mathcal{O}(\alpha/\epsilon)$ they achieve the same deterministic guarantees as in the insertion-only case.
- For Count-Min and Count-Sketch, we implement sampling-based optimizations by [JW18] that reduce the space dependence from $\mathcal{O}(\log n)$ to $\mathcal{O}(\log \alpha)$, and we empirically evaluate the trade-offs between sampling rate and accuracy.
- We conduct extensive experiments on synthetic data (Zipfian and uniform distributions) and a real-world CAIDA network trace, varying the stream length, $\alpha \in \{1.5, 2, 4, 8\}$, and deletion patterns. We measure the Average Relative Error (ARE), the Average Normalized Absolute Error (ANAE), the F_1 Score for Heavy Hitters, and compare the four extended algorithms against each other.
- Motivated by the experimental trade-offs-where counter-based methods excel in accuracy but scale linearly with α, while sketches are space-efficient but less accurate for heavy hitters-we propose a novel conceptual design: a *Dyadic Count-Min Hierarchy with an Adaptive Counter Cache*. This structure combines a tree of sampled Count-Min sketches (achieving space logarithmic in α) with a small deterministic cache that preserves the accuracy of counter-based methods for the heaviest items. The design offers a promising path toward near-optimal space-accuracy trade-offs in future streaming systems.

The remainder of this report is organized as follows. Section 2 details our extensions and their theoretical analysis. Section 3 describes the experimental setup, datasets, and discusses the experimental results. Section 4 outlines the proposed hybrid design. Section 5 concludes and suggests directions for future work.

2 Algorithm Extensions and Analyses

2.1 A Review of Standard Algorithm Formulations

We first describe the four frequency estimation algorithms in their classic formulations. Each algorithm maintains a compact sketch of the stream and supports two operations: `update` (process an incoming element) and `query` (return an estimate of an element's frequency). The descriptions of algorithms in this section focus on the data structures, update rules, and theoretical guarantees for insertion-only streams (for Misra-Gries and Space-Saving) or the standard turnstile model (for Count-Min and Count-Sketch). In the next subsection, we will extend these algorithms to the strict turnstile α-bounded deletion model.

2.1.1 Misra-Gries (Frequent) Algorithm

The Misra-Gries algorithm maintains a set of at most k counters, each storing an item identifier and its estimated frequency. The algorithm processes the stream in a single pass. When an item e arrives, if e is already monitored, its counter is incremented by one. If e is not monitored and the set contains fewer than k counters, a new counter for e is created with value one. Otherwise, when the set is full and e is absent, every existing counter is decremented by one, and any counter that reaches zero is removed. For a query, if e is present in the set, its counter value is returned; otherwise zero is returned.

With $k = \lceil 1/\epsilon \rceil - 1$ counters, the algorithm guarantees that $0 \leq f_e - \hat{f}_e \leq \epsilon N$ for every item e, where N is the total number of insertions [MG82]. The estimate never overestimates the true frequency.

Algorithm 1: Misra-Gries (Frequent) Algorithm

Data: Stream of elements $e_1, e_2, \ldots$ from universe $[m]$, parameter $\epsilon \in (0, 1)$.
Result: Estimated frequencies $\hat{f}_e$ for every e.

let S be a set of pairs $\langle e, c \rangle$, initially empty;
for *each element e in the stream* **do**
  if *there exists a pair $\langle e, c \rangle \in S$* **then**
    update the pair to $\langle e, c + 1 \rangle$;
  else if $|S| < \lceil 1/\epsilon \rceil - 1$ **then**
    add $\langle e, 1 \rangle$ to S;
  else
    for *each pair $\langle e', c \rangle \in S$* **do**
      decrement c by 1;
      if $c = 0$ **then**
        remove the pair from S;
return $\hat{f}_e = \begin{cases} c & \text{if } \langle e, c \rangle \in S \\ 0 & \text{otherwise} \end{cases}$;

2.1.2 Space-Saving Algorithm

Space-Saving also uses $k = \mathcal{O}(1/\epsilon)$ counters but employs a different replacement strategy to keep track of the most frequent items. It maintains a min-heap (or a set of buckets) to efficiently retrieve the counter with the smallest estimated frequency. When an item e arrives, if e is already monitored, its counter is incremented. If e is not monitored and fewer than k counters are used, a new counter for e is created with value one. If the set is full, the counter with the smallest value $c_{\min}$ is evicted and replaced by a counter for e with value $c_{\min} + 1$. For a query, the estimated frequency of e is its counter value if it is monitored, otherwise zero.

With $k = \lceil 1/\epsilon \rceil$, Space-Saving guarantees that for every item e, $f_e \leq \hat{f}_e \leq f_e + \epsilon N$ [MAEA05]; that is, the estimate may overestimate but the additive error is bounded by $\epsilon \|\mathbf{f}\|$.

Algorithm 2: Space-Saving Algorithm

Data: Stream of elements $e_1, e_2, \ldots$ from universe $[m]$, parameter $\epsilon \in (0, 1)$.
Result: Estimated frequencies $\hat{f}_e$ for every e.

let S be a set of pairs $\langle e, c \rangle$, initially empty;
for *each element e in the stream* **do**
  if $\langle e, c \rangle \in S$ **then**
    update the pair to $\langle e, c + 1 \rangle$;
  else if $|S| < \lceil 1/\epsilon \rceil$ **then**
    add $\langle e, 1 \rangle$ to S;
  else
    let $\langle e_{\min}, c_{\min} \rangle$ be the pair with the smallest count in S;
    remove $\langle e_{\min}, c_{\min} \rangle$ from S;
    add $\langle e, c_{\min} + 1 \rangle$ to S;
return $\hat{f}_e = \begin{cases} c & \text{if } \langle e, c \rangle \in S \\ 0 & \text{otherwise} \end{cases}$;

2.1.3 Count-Min Sketch

The Count-Min Sketch is a probabilistic data structure that uses d pairwise independent hash functions $h_1, \ldots, h_d$, each mapping items to a range of w buckets. The sketch is a $d \times w$ array of counters initially set to zero. For an update (e, Δ) (where Δ can be positive or negative), the algorithm adds Δ to $C[i][h_i(e)]$ for each row i. For a query, the estimated frequency of e is $\hat{f}_= \min_i C[i][h_i(e)]$.

With parameters $w = \lceil 3/\epsilon \rceil$ and $d = \lceil \log_3(1/\delta) \rceil$, the sketch guarantees $\hat{f}_e \geq f_e$ always and $\Pr[\hat{f}_e - f_e \leq \epsilon \|\mathbf{f}\|_1] \geq 1 - \delta$ [CM05]. The total space required is $\mathcal{O}\left(\log(1/\delta) \log m/\epsilon\right)$ bits, where m is the number of elements in the universe.

Algorithm 3: Count-Min Sketch

Data: Stream of updates (e, Δ) where $e \in [m]$, $\Delta \in \mathbb{Z}$; parameters d, w.
Result: Estimated frequency $\hat{f}_e$ for any e.

```
for i = 1 to d do
    Choose a pairwise independent hash function h_i : [m] → [w];
Initialize a d × w array C with all zeros;
for each update (e, Δ) do
    for i = 1 to d do
        C[i][h_i(e)] ← C[i][h_i(e)] + Δ;
return f̂_e = min_{i=1}^{d} C[i][h_i(e)];
```

2.1.4 Count-Sketch

Count-Sketch [CCFC02] similarly employs a $d \times w$ array of counters, but associates two hash functions per row: h_i selects a bucket and $s_i : [m] \to \{-1, +1\}$ provides a random sign. For an update (e, Δ), the algorithm computes $j_i = h_i(e)$ and $s = s_i(e)$ for each row i, then updates $C[i][j_i] \leftarrow C[i][j_i] + s \cdot \Delta$. To answer a query for item e, the algorithm computes $v_i = s_i(e) \cdot C[i][h_i(e)]$ for each row and returns the median of the v_i values.

Count-Sketch provides an unbiased estimator and guarantees $\Pr[|\hat{f}_e - f_e| \leq \epsilon \|\mathbf{f}\|_2] \geq 1 - \delta$ when $w = \Theta(1/\epsilon^2)$ and $d = \Theta(\log(n/\delta))$ [Vak25]. The space complexity is $\mathcal{O}\big(\log(n/\delta) \log n/\epsilon^2\big)$ bits.

Algorithm 4: Count-Sketch

Data: Stream of updates (e, Δ) where $e \in [m]$, $\Delta \in \mathbb{Z}$; parameters d, w
Result: Estimated frequency $\hat{f}_e$ for any e

```
for i = 1 to d do
    choose pairwise independent hash functions h_i : [m] → [w] (bucket index) and
      s_i : [m] → {+1, −1} (sign);
initialize a d × w array C with all zeros;
for each update (e, Δ) do
    for i = 1 to d do
        C[i][h_i(e)] ← C[i][h_i(e)] + s_i(e) · Δ;
return f̂_e = median(v_1, ..., v_d), where v_i = s_i(e) · C[i][h_i(e)] for i = 1, ..., d;
```

2.2 Extensions for Counter-Based Algorithms and Analyses

In this subsection we adapt two classical deterministic counter-based algorithms, Misra-Gries and Space-Saving, to the α-bounded deletion model. The key idea is to maintain two separate summaries: one that tracks insertions and another that tracks deletions. Each summary is updated using the original algorithm on the corresponding non-negative substream, and the estimated net frequency of an item is obtained by taking the maximum of the difference between the two estimates and zero. We show that under the α-bounded condition, this "double" approach preserves the strong deterministic guarantees of the original methods. Specifically, with $m_I + m_D = \mathcal{O}(\alpha/\epsilon)$ counters we achieve an $\epsilon\|\mathbf{f}\|_1$ frequency estimation error and correctly identify all ϵ-heavy hitters. The following two sections present the Double Misra-Gries± and Double Space-Saving± algorithms together with their correctness proofs.

2.2.1 The Double Misra-Gries± Algorithm

We prove that the Double Misra-Gries± algorithm solves the deterministic frequency estimation and heavy hitters problems in the bounded deletion model. With $m_I + m_D = \mathcal{O}(\alpha/\epsilon)$ counters, we obtain the following guarantee.

Theorem 2.1 (Correctness of Double Misra-Gries±)**.** *Let $\mathcal{S}$ be an α-bounded deletion stream. If the algorithm is run with*

$$m_I = \left\lceil \frac{2(\alpha+1)}{\epsilon} \right\rceil - 1, \qquad m_D = \left\lceil \frac{2(\alpha-1)}{\epsilon} \right\rceil - 1,$$

then for every item e,

$$|f_e - \hat{f}_e| \leq \epsilon \|\mathbf{f}\|_1,$$

where $\|\mathbf{f}\|_1 = \sum_x f(x)$ is the total net frequency. Moreover, the algorithm correctly identifies all ϵ-heavy hitters (items with $f_e \geq \epsilon\|\mathbf{f}\|_1$).

Proof. From $\|\mathbf{I}\|_1 - \|\mathbf{D}\|_1 = \|\mathbf{f}\|_1$ and the α-bounded property that $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 \le \alpha\|\mathbf{f}\|_1$, we obtain

$$\|\mathbf{I}\|_1 \le \frac{\alpha+1}{2}\|\mathbf{f}\|_1, \qquad \|\mathbf{D}\|_1 \le \frac{\alpha-1}{2}\|\mathbf{f}\|_1.$$

For any item e, let I_e and D_e denote its true insertion and deletion counts, and let $\hat{I}_e$, $\hat{D}_e$ be the estimates returned by $\mathcal{S}_I$ and $\mathcal{S}_D$, respectively.

By the classic property of Misra-Gries, with m_I counters we have

$$0 \le \hat{I}_e \le I_e, \qquad I_e - \hat{I}_e \le \frac{\|\mathbf{I}\|_1}{m_I+1},$$

and similarly

$$0 \le \hat{D}_e \le D_e, \qquad D_e - \hat{D}_e \le \frac{\|\mathbf{D}\|_1}{m_D+1}.$$

The query returns $\hat{f}_e = \max(\hat{I}_e - \hat{D}_e, 0)$. We claim that

$$|f_e - \hat{f}_e| \le (I_e - \hat{I}_e) + (D_e - \hat{D}_e).$$

Indeed, consider two possibilities:

Case 1: If $\hat{I}_e - \hat{D}_e \ge 0$, then $\hat{f}_e = \hat{I}_e - \hat{D}_e$ and the triangle inequality yields

$$|f_e - \hat{f}_e| = |(I_e - D_e) - (\hat{I}_e - \hat{D}_e)| \le |I_e - \hat{I}_e| + |D_e - \hat{D}_e| = (I_e - \hat{I}_e) + (D_e - \hat{D}_e).$$

Case 2: If $\hat{I}_e - \hat{D}_e < 0$, then $\hat{f}_e = 0$. Because $\hat{D}_e > \hat{I}_e$ and $\hat{D}_e \le D_e$, we have $D_e > \hat{I}_e$. Hence

$$f_e = I_e - D_e \le I_e - \hat{I}_e \le (I_e - \hat{I}_e) + (D_e - \hat{D}_e).$$

Thus the inequality holds as well.

Therefore,

$$|f_e - \hat{f}_e| \le (I_e - \hat{I}_e) + (D_e - \hat{D}_e) \le \frac{\|\mathbf{I}\|_1}{m_I+1} + \frac{\|\mathbf{D}\|_1}{m_D+1}.$$

Now choose $m_I + 1 = \lceil 2(\alpha+1)/\epsilon \rceil$ and $m_D + 1 = \lceil 2(\alpha-1)/\epsilon \rceil$. Then

$$\frac{\|\mathbf{I}\|_1}{m_I+1} + \frac{\|\mathbf{D}\|_1}{m_D+1} \le \frac{(\alpha+1)\|\mathbf{f}\|_1}{2(\alpha+1)/\epsilon} + \frac{(\alpha-1)\|\mathbf{f}\|_1}{2(\alpha-1)/\epsilon} = \frac{\epsilon}{2}\|\mathbf{f}\|_1 + \frac{\epsilon}{2}\|\mathbf{f}\|_1 = \epsilon\|\mathbf{f}\|_1.$$

The total number of counters is $m_I + m_D \le \frac{2(\alpha+1)}{\epsilon} + \frac{2(\alpha-1)}{\epsilon} = \frac{4\alpha}{\epsilon} = \mathcal{O}(\alpha/\epsilon)$. Thus, Double Misra-Gries± solves the frequency estimation problem with the claimed space.

For heavy hitters, if $f_e \ge \epsilon\|\mathbf{f}\|_1$, then e must appear in $\mathcal{S}_I$; otherwise $\hat{I}_e = 0$ and $\hat{f}_e = 0$, contradicting the error bound above. Hence, all heavy hitters are monitored in $\mathcal{S}_I$, and reporting all items with $\hat{f}_e \ge \epsilon\|\mathbf{f}\|_1$ (or simply all items in $\mathcal{S}_I$) solves the heavy hitters problem. □

Algorithm 5: Double Misra-Gries± for Bounded Deletions

Data: Stream of operations (e_t, op_t) where $op_t \in \{\text{insertion}, \text{deletion}\}$, parameters m_I, m_D (maximum counters for insert and delete summaries).
Result: Estimated frequencies $\hat{f}_e$ for every e.

initialize two summaries $\mathcal{S}_I$ and $\mathcal{S}_D$ as empty sets of pairs $\langle e, cnt\rangle$;
for *each operation* (e, op) *in the stream* **do**
  if *op is an insertion* **then**
    if $\exists\langle e, cnt\rangle \in \mathcal{S}_I$ **then**
      update to $\langle e, cnt + 1\rangle$;
    else if $|\mathcal{S}_I| < m_I$ **then**
      add $\langle e, 1\rangle$ to $\mathcal{S}_I$;
    else
      for *each* $\langle x, cnt_x\rangle \in \mathcal{S}_I$ **do**
        $cnt_x \leftarrow cnt_x - 1$;
        if $cnt_x = 0$ **then**
          remove $\langle x, 0\rangle$ from $\mathcal{S}_I$;
      add $\langle e, 1\rangle$ to $\mathcal{S}_I$;
  else
    if $\exists\langle e, cnt\rangle \in \mathcal{S}_D$ **then**
      update to $\langle e, cnt + 1\rangle$;
    else if $|\mathcal{S}_D| < m_D$ **then**
      add $\langle e, 1\rangle$ to $\mathcal{S}_D$;
    else
      for *each* $\langle x, cnt_x\rangle \in \mathcal{S}_D$ **do**
        $cnt_x \leftarrow cnt_x - 1$;
        if $cnt_x = 0$ **then**
          remove $\langle x, 0\rangle$ from $\mathcal{S}_D$;
      add $\langle e, 1\rangle$ to $\mathcal{S}_D$;

return $\hat{f}_e = \max\big(query_{\mathcal{S}_I}(e) - query_{\mathcal{S}_D}(e),\ 0\big)$, *where* $query_{\mathcal{S}}(e) = \begin{cases} cnt & \text{if } \langle e, cnt\rangle \in \mathcal{S} \\ 0 & \text{otherwise} \end{cases}$;

2.2.2 The Double Space-Saving± Algorithm

We prove that the Double SpaceSaving± algorithm also solves the deterministic frequency estimation and heavy hitters problems in the bounded deletion model.

With $m_I + m_D = \mathcal{O}(\alpha/\epsilon)$ counters, we have the following guarantee for the Double SpaceSaving± algorithm.

Theorem 2.2 (Correctness of Double SpaceSaving±)**.** *Let $\mathcal{S}$ be an α-bounded deletion stream. If the algorithm is run with $m_I = \lceil \frac{2(\alpha+1)}{\epsilon} \rceil$ and $m_D = \lceil \frac{2(\alpha-1)}{\epsilon} \rceil$ counters, then for every item e,*

$$|f_e - \hat{f}_e| \le \epsilon \|\mathbf{f}\|_1.$$

Moreover, the algorithm correctly identifies all ϵ-heavy hitters (items with $f_e \ge \epsilon\|\mathbf{f}\|_1$).

Proof. From $\|\mathbf{I}\|_1 - \|\mathbf{D}\|_1 = \|\mathbf{f}\|_1$ and $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 \le \alpha\|\mathbf{f}\|_1$, we obtain

$$\|\mathbf{I}\|_1 \le \frac{\alpha+1}{2}\|\mathbf{f}\|_1, \qquad \|\mathbf{D}\|_1 \le \frac{\alpha-1}{2}\|\mathbf{f}\|_1.$$

For any item e, denote by $\hat{I}_e$ the estimate from $\mathcal{S}_I$ and $\hat{D}_e$ the estimate from $\mathcal{S}_D$. By the known properties of SpaceSaving,

$$|I_e - \hat{I}_e| \le \frac{\|\mathbf{I}\|_1}{m_I}, \qquad |D_e - \hat{D}_e| \le \frac{\|\mathbf{D}\|_1}{m_D}.$$

Using the triangle inequality and $f_e = I_e - D_e$,

$$|f_e - \hat{f}_e| \le |I_e - \hat{I}_e| + |D_e - \hat{D}_e| \le \frac{\|\mathbf{I}\|_1}{m_I} + \frac{\|\mathbf{D}\|_1}{m_D}.$$

Algorithm 6: Double SpaceSaving± for Bounded Deletions

Data: Stream of operations (e_t, op_t) where $op_t \in \{\text{insertion}, \text{deletion}\}$, parameters m_I, m_D (maximum counters for insert and delete summaries).
Result: Estimated frequencies $\hat{f}_e$ for every e.

initialize two summaries $\mathcal{S}_I$ and $\mathcal{S}_D$ as empty sets of pairs $\langle e, cnt\rangle$;
for *each operation* (e, op) *in the stream* **do**
  if *op is an insertion* **then**
    if $\exists \langle e, cnt\rangle \in \mathcal{S}_I$ **then**
      update to $\langle e, cnt + 1\rangle$;
    else if $|\mathcal{S}_I| < m_I$ **then**
      add $\langle e, 1\rangle$ to S_I;
    else
      find the item x with smallest count $c_{\min}$ in $\mathcal{S}_I$;
      remove $\langle x, c_{\min}\rangle$ from $\mathcal{S}_I$;
      add $\langle e, c_{\min} + 1\rangle$ to $\mathcal{S}_I$;
  else
    if $\exists \langle e, cnt\rangle \in \mathcal{S}_D$ **then**
      update to $\langle e, cnt + 1\rangle$;
    else if $|\mathcal{S}_D| < m_D$ **then**
      add $\langle e, 1\rangle$ to $\mathcal{S}_D$;
    else
      find the item y with smallest count $d_{\min}$ in $\mathcal{S}_D$;
      remove $\langle y, d_{\min}\rangle$ from $\mathcal{S}_D$;
      add $\langle e, d_{\min} + 1\rangle$ to $\mathcal{S}_D$;
return $\hat{f}_e = \max\big(query_{\mathcal{S}_I}(e) - query_{\mathcal{S}_D}(e),\, 0\big)$, *where* $query_S(e) = \begin{cases} cnt & \text{if } \langle e, cnt\rangle \in S \\ 0 & \text{otherwise} \end{cases}$;

Choose $m_I = \lceil \frac{2(\alpha+1)}{\epsilon} \rceil$ and $m_D = \lceil \frac{2(\alpha-1)}{\epsilon} \rceil$. Then,

$$\frac{\|\mathbf{I}\|_1}{m_I} + \frac{\|\mathbf{D}\|_1}{m_D} \leq \frac{\alpha\|\mathbf{f}\|_1}{2\alpha/\epsilon} + \frac{(\alpha-1)\|\mathbf{f}\|_1}{2(\alpha-1)/\epsilon} = \frac{\epsilon\|\mathbf{f}\|_1}{2} + \frac{\epsilon\|\mathbf{f}\|_1}{2} = \epsilon\|\mathbf{f}\|_1.$$

Thus, for all element e, $|f_e - \hat{f}_e| \leq \epsilon\|\mathbf{f}\|_1$, so Double SpaceSaving± solves the frequency estimation problem using $\mathcal{O}(\alpha/\epsilon)$ counters.

Moreover, if e is a heavy hitter, i.e., $f_e \geq \epsilon\|\mathbf{f}\|_1$, it must be monitored in $\mathcal{S}_I$. Indeed, if e were not in $\mathcal{S}_I$, then $\hat{I}_e = 0$ and therefore $\hat{f}_e = 0$, giving an error $|f_e - 0| \geq \epsilon\|\mathbf{f}\|_1$, contradicting the above bound. Hence all heavy hitters appear in $\mathcal{S}_I$, and reporting all items with $\hat{f}_e \geq \epsilon\|\mathbf{f}\|_1$ (or simply all monitored items in $\mathcal{S}_I$) solves the heavy hitters problem. □

2.3 Extensions for Sketch-Based Algorithms with the Stream Downsampling Technique

The standard Count-Min Sketch and Count-Sketch supports deletions directly. To exploit the α-property, we process only a small random sample of the stream, following the framework of Jayaram and Woodruff [JW18]. The key idea is to process only a small random sample of the stream, scale the sampled counts, and then run the standard sketch on the sampled substream. As the α-property guarantees that the total number of updates n is at most $\alpha\|\mathbf{f}\|_1$, a carefully chosen sample size $\mathcal{O}(\alpha^2/\epsilon^3)$ suffices to preserve frequencies up to additive $\epsilon\|\mathbf{f}\|_1$. This reduces the space from $\mathcal{O}(\log n)$ to $\mathcal{O}(\log \alpha)$ per counter.

To kickstart the discussion for the following sections, we will make use of the sampling lemma from Jayaram and Woodruff [JW18].

Lemma 2.1 (Sampling Lemma, Lemma 1 of [JW18])**.** *Let the stream satisfy the L_1 α-property with final L_1 norm $\|\mathbf{f}\|_1$ and let $\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1$ be the total number of updates. If each update is sampled independently with probability*

$$p > \frac{\alpha^2 \log(1/\delta)}{\epsilon^3(\|\mathbf{I}\|_1 + \|\mathbf{D}\|_1)},$$

and the sampled frequencies are scaled by $1/p$, then with probability at least $1-\delta$ the scaled sampled frequency vector f^* satisfies

$$|f_e^* - f_e| \leq \epsilon \|\mathbf{f}\|_1 \qquad \text{for all items } e.$$

2.3.1 Count-Min Sketch with Uniform Sampling

We propose a simple uniform sampling extension inspired by [JW18] for the Count-Min Sketch that exploits the α-property. Let $w = \lceil 9/\epsilon \rceil$ be the number of columns. Set the number of rows $d = \lceil \log_3(2/\delta) \rceil$ and the sampling probability to be $p > \alpha^2 \log(2/\delta)/(27m\epsilon^3)$, we have the following guarantee:

Theorem 2.3. *For any $\epsilon, \delta \in (0,1)$, the Extended Count-Min Sketch with the parameters set as the above satisfies*

$$\Pr\big[|\hat{f}_e - f_e| \leq \epsilon \|\mathbf{f}\|_1 \text{ for all } e\big] \geq 1 - \delta.$$

The space used is $\mathcal{O}\big(\frac{1}{\epsilon} \log \frac{1}{\delta} \log \frac{\alpha}{\epsilon}\big)$ bits.

Proof. Let $f_{e,\text{sample}}$ be the net frequency of e in the sampled stream, with the scaled frequency being $f_e^* = f_{e,\text{sample}}/p$. By the sampling lemma (Lemma 2.1, [JW18]), with probability at least $1 - \delta/2$ the scaled sampled frequencies satisfy $|f_e^* - f_e| \leq (\epsilon/3)\|\mathbf{f}\|_1$ for all e. Denote this event as $\mathcal{E}_1$.

The standard Count-Min guarantee then ensures that with probability $1 - \delta/2$, $\hat{f}_{e,\text{sample}} \geq f_{e,\text{sample}}$ and $\hat{f}_{e,\text{sample}} - f_{e,\text{sample}} \leq (\epsilon/3)\|\mathbf{f}_{\text{sample}}\|_1$. Dividing by p, we have $\hat{f}_e - f_e^* \leq (\epsilon/3)\|\mathbf{f}^*\|_1$. Denote this event as $\mathcal{E}_2$.

Under $\mathcal{E}_1$ and $\mathcal{E}_2$, we have

$$\|\mathbf{f}^*\|_1 \leq (1 + \epsilon/3)\|\mathbf{f}\|_1$$

and

$$\hat{f}_e - f_e^* \leq (\epsilon/3)\|\mathbf{f}^*\|_1.$$

Hence,

$$\hat{f}_e - f_e^* \leq (\epsilon/3)(1 + \epsilon/3)\|\mathbf{f}\|_1 \leq (2\epsilon/3)\|\mathbf{f}\|_1.$$

Moreover, we have

$$\hat{f}_e \geq f_e^*.$$

Thus,

$$\hat{f}_e - f_e = (\hat{f}_e - f_e^*) + (f_e^* - f_e) \leq (2\epsilon/3)\|\mathbf{f}\|_1 + (\epsilon/3)\|\mathbf{f}\|_1 = \epsilon\|\mathbf{f}\|_1,$$

$$\hat{f}_e \geq f_e^* \geq f_e - (\epsilon/3)\|\mathbf{f}\|_1 \Rightarrow \hat{f}_e - f_e \geq -(\epsilon/3)\|\mathbf{f}\|_1,$$

and therefore

$$|\hat{f}_e - f_e| \leq \epsilon\|\mathbf{f}\|_1.$$

By the union bound,

$$\Pr[\mathcal{E}_1 \cap \mathcal{E}_2] \geq 1 - \frac{\delta}{2} - \frac{\delta}{2} = 1 - \delta.$$

The expected number of sampled updates is $pm = \mathcal{O}(\alpha^2/\epsilon^3)$ with high probability (which can be bounded by the Chernoff Bound, and we can set p to be large enough). The total sum of the scaled counter values is at most the total scaled weight of the sampled stream, which is $\mathcal{O}(\alpha/\epsilon)$. Thus each counter can be stored using $\mathcal{O}(\log(\alpha/\epsilon))$ bits (we can terminate if a counter gets too large). The sketch has $d \times w = \mathcal{O}\big(\frac{1}{\epsilon} \log \frac{1}{\delta}\big)$ counters, so the total space is $\mathcal{O}\big(\frac{1}{\epsilon} \log \frac{1}{\delta} \log \frac{\alpha}{\epsilon}\big)$ bits. The total space is $\mathcal{O}\big(\frac{1}{\epsilon} \log \frac{1}{\delta} \log \frac{\alpha}{\epsilon}\big)$ bits. □

Algorithm 7: Extended Count-Min Sketch for α-Bounded Deletion Streams

Data: Stream of updates (e, Δ) where $e \in [m]$, $\Delta \in \mathbb{Z}$; parameters ϵ, δ, α, and an upper bound M on the total number of updates.
Result: Estimated frequency $\hat{f}_e$ for any e.

compute sampling probability $p \leftarrow \min\big(1,\ \alpha^2 \log(2/\delta)/(27M\epsilon^3)\big)$;
initialize sketch with $w \leftarrow \lceil 9/\epsilon \rceil$ and $d \leftarrow \lceil \log_3(2/\delta) \rceil$;
for $i = 1$ *to* d **do**
    choose a pairwise independent hash function $h_i : [m] \to [w]$
initialize a $d \times w$ array C with all zeros;
for *each update* (e, Δ) **do**
    accept e with the probability p;
    if e *is accepted* **then**
        for $i = 1$ *to* d **do**
            $j \leftarrow h_i(e)$;
            $C[i][j] \leftarrow C[i][j] + \frac{\Delta}{p}$;
return $\hat{f}_e = \min_{i=1}^{d} C[i][h_i(e)]$;

2.3.2 Count-Sketch with Uniform Sampling

We also present a simple uniform sampling extension for the Count-Sketch that exploits the α-property to reduce the space dependency from $\log n$ to $\log \alpha$. The algorithm is easy to implement and achieves an additive error guarantee of $\mathcal{O}(\epsilon\|\mathbf{f}\|_1)$ with high probability.

Let $w = \lceil 27/\epsilon^2 \rceil$ be the number of columns. Set the number of rows $d = \lceil \log_3(2/\delta) \rceil$ and the sampling probability to be $p > \alpha^2 \log(2/\delta)/(27m\epsilon^3)$, we have the following guarantee:

Theorem 2.4. *Under the strict turnstile α-property, with probability at least $1 - \delta$ the estimate $\hat{f}_e$ given by the Count-Sketch satisfies*

$$|\hat{f}_e - f_e| \leq \epsilon\|\mathbf{f}\|_1$$

for every item e. The total space used is

$$\mathcal{O}\left(\frac{1}{\epsilon^2} \log \frac{1}{\delta} \log \frac{\alpha}{\epsilon}\right)$$

bits.

Proof. Let $\mathbf{f}^*$ be the frequency vector of the sampled substream scaled by $1/p$. Formally, for each item e, let $f_{e,\text{sample}}$ be the number of times e appears in the sampled substream (with sign, i.e., $+1$ for insertions, -1 for deletions). Then $f_e^* = f_{e,\text{sample}}/p$. With the chosen p, Lemma 2.1 guarantees that with probability at least $1 - \delta/2$,

$$|f_e^* - f_e| \leq \frac{\epsilon}{3}\|\mathbf{f}\|_1 \qquad \text{for all } e.$$

The sampled substream has expected length $pn = \mathcal{O}(\alpha^2/\epsilon^3)$. By a Chernoff bound, its actual length is $\mathcal{O}(\alpha^2/\epsilon^3)$ with probability at least $1 - \delta/2$, conditioned on this event.

Run Count-Sketch on this sampled substream *without scaling*. Let $\hat{f}_{e,\text{sample}}$ be the estimate obtained from the sketch (i.e., the median of the row estimates). The classic Count-Sketch guarantee ensures that

$$\Pr\left[|\hat{f}_{e,\text{sample}} - f_{e,\text{sample}}| \leq \frac{\epsilon}{3}\|\mathbf{f}_{\text{sample}}\|_2\right] \geq 1 - \frac{\delta}{2}.$$

Now note that $\|\mathbf{f}_{\text{sample}}\|_2 \leq \|\mathbf{f}_{\text{sample}}\|_1$. Since $f_{e,\text{sample}} = p\, f_e^*$ and by the Sampling Lemma $|f_e^*| \leq |f_e| + \frac{\epsilon}{3}\|\mathbf{f}\|_1$, we have

$$\|\mathbf{f}_{\text{sample}}\|_1 = p\|\mathbf{f}^*\|_1 \leq p\Big(1 + \frac{\epsilon}{3}\Big)\|\mathbf{f}\|_1.$$

Therefore,

$$\frac{\epsilon}{3}\|\mathbf{f}_{\text{sample}}\|_2 \leq \frac{\epsilon}{3}\|\mathbf{f}_{\text{sample}}\|_1 \leq \frac{\epsilon}{3}p\Big(1 + \frac{\epsilon}{3}\Big)\|\mathbf{f}\|_1 \leq \frac{2\epsilon}{3}p\|\mathbf{f}\|_1$$

for $\epsilon \leq 1$. Hence with probability $1 - \delta/2$,

$$|\hat{f}_{e,\text{sample}} - f_{e,\text{sample}}| \leq \frac{2\epsilon}{3}p\|\mathbf{f}\|_1.$$

Recall that $\hat{f}_e = \hat{f}_{e,\mathrm{CS}}/p$ and $f_e^* = f_{e,\mathrm{sample}}/p$. Then

$$|\hat{f}_e - f_e^*| = \frac{|\hat{f}_{e,\mathrm{CS}} - f_{e,\mathrm{sample}}|}{p} \leq \frac{2\epsilon}{3}\|\mathbf{f}\|_1.$$

Using the Sampling Lemma,

$$|f_e^* - f_e| \leq \frac{\epsilon}{3}\|\mathbf{f}\|_1.$$

By the triangle inequality,

$$|\hat{f}_e - f_e| \leq |\hat{f}_e - f_e^*| + |f_e^* - f_e| \leq \frac{2\epsilon}{3}\|\mathbf{f}\|_1 + \frac{\epsilon}{3}\|\mathbf{f}\|_1 \leq \epsilon\|\mathbf{f}\|_1$$

for $\epsilon \leq 1$. The union bound over the two events gives success probability at least $1 - \delta$.

The sampled substream contains $\mathcal{O}(\alpha^2/\epsilon^3)$ updates with high probability. Each counter in the Count-Sketch stores the sum of signed updates that fall into that bucket. Since each update contributes ± 1 to a single counter per row, every counter value is bounded by the total number of updates in the sampled substream, i.e., $\mathcal{O}(\alpha^2/\epsilon^3)$. Thus each counter requires $\mathcal{O}(\log(\alpha/\epsilon))$ bits. The sketch has $d = \mathcal{O}(\log(1/\delta))$ rows and $w = \mathcal{O}(1/\epsilon^2)$ columns, so the total space is

$$\mathcal{O}\big(\log(1/\delta)\log(\alpha/\epsilon)/\epsilon^2\big)$$

bits.

□

Algorithm 8: Extended Count-Sketch for α-Bounded Deletion Streams

Data: Stream of updates (e, Δ) where $e \in [m]$, $\Delta \in \mathbb{Z}$; parameters ϵ, δ, α, and an upper bound M on the total number of updates.
Result: Estimated frequency $\hat{f}_e$ for any e.

compute sampling probability $p \leftarrow \min\left(1, \frac{\alpha^2 \log(2/\delta)}{27M\epsilon^3}\right)$;
initialize sketch with $w \leftarrow \lceil 27/\epsilon^2 \rceil$ and $d \leftarrow \lceil \log_3(2/\delta) \rceil$;
for $i = 1$ *to* d **do**
  choose pairwise independent hash functions $h_i : [m] \to [w]$ and $s_i : [m] \to \{-1, +1\}$
initialize a $d \times w$ array C with all zeros;
for *each update* (e, Δ) **do**
  accept the update with probability p;
  if *accepted* **then**
    for $i = 1$ *to* d **do**
      $j \leftarrow h_i(e)$;
      $C[i][j] \leftarrow C[i][j] + s_i(e) \cdot \Delta$;
return $\hat{f}_e = \mathrm{median}_{i=1}^{d}\left(s_i(e) \cdot C[i][h_i(e)]\right) / p$;

2.3.3 Count-Sketch with `CSSampSim` Data Structure

We now present the `CSSampSim` data structure, an extension of Count-Sketch for the strict turnstile α-bounded deletion model. `CSSampSim` simulates running a Count-Sketch on a uniform sample of the stream, but uses a clever adaptive sampling method to avoid storing the entire sample. The algorithm maintains a fixed number of counters that are repeatedly halved in a "lazy" manner, mimicking exponential sampling.

Theorem 2.5. *For any $\epsilon \in (0, 1)$, there exists a one-pass algorithm that solves the ϵ-heavy hitters problem for strict turnstile α-property streams with high probability (probability $1 - 1/\mathrm{poly}(n)$) using space*

$$\mathcal{O}\left(\frac{1}{\epsilon}\log n \log\frac{\alpha\log n}{\epsilon}\right)$$

bits.

Proof Sketch. We use the `CSSampSim` data structure given above [JW18]. We refer the readers to [JW18] for the error-free proof.

Set the number of columns to $k = 32/\epsilon$ and choose $\epsilon' = \epsilon/32$. Instantiate `CSSampSim` with k columns, $d = \mathcal{O}(\log n)$ rows, and parameters

$$S = \Theta\left(\frac{\alpha^2}{\epsilon^2}T^2\log n\right), \qquad T = \frac{4}{\epsilon^2} + \log n.$$

After processing the stream, `CSSampSim` returns estimates y_i^* such that by Lemma 2 of [JW18], with high probability

$$|y_i^* - f_i| \le 2\left(\frac{1}{\sqrt{k}}\,\|\mathbf{f}_{-k}\|_2 + \epsilon'\|\mathbf{f}\|_1\right).$$

Since the stream is strict turnstile, we can maintain an exact counter of the net sum of all updates, which gives $\|\mathbf{f}\|_1$ using $\mathcal{O}(\log n)$ bits.

We now bound $\|\mathbf{f}_{-k}\|_2$. For the tail (the set of coordinates not among the k largest), each such coordinate i satisfies $|f_i| \le \|\mathbf{f}\|_1/k$ (otherwise the k largest would be at least that large, contradicting the sum of the k largest being at most $\|\mathbf{f}\|_1$). Hence,

$$\|\mathbf{f}_{-k}\|_2 = \sqrt{\sum_{i\in\text{tail}} f_i^2} \le \sqrt{\left(\sum_{i\in\text{tail}} |f_i|\right)\cdot \max_{i\in\text{tail}} |f_i|} \le \sqrt{\|\mathbf{f}\|_1 \cdot \frac{\|\mathbf{f}\|_1}{k}} = \frac{\|\mathbf{f}\|_1}{\sqrt{k}}.$$

Together with $k = 32/\epsilon$ and $\epsilon' = \epsilon/32$, it yields that

$$|y_i^* - f_i| \le 2\left(\frac{1}{\sqrt{k}}\cdot\frac{\|\mathbf{f}\|_1}{\sqrt{k}} + \frac{\epsilon}{32}\|\mathbf{f}\|_1\right) = 2\left(\frac{\|\mathbf{f}\|_1}{k} + \frac{\epsilon}{32}\|\mathbf{f}\|_1\right) = 2\left(\frac{\epsilon}{32}\|\mathbf{f}\|_1 + \frac{\epsilon}{32}\|\mathbf{f}\|_1\right) = \frac{\epsilon}{8}\|\mathbf{f}\|_1.$$

The algorithm reports every item i for which $y_i^* \ge \frac{3\epsilon}{4}\|\mathbf{f}\|_1$.

- If $f_i \ge \epsilon\|f\|_1$, then $y_i^* \ge f_i - \frac{\epsilon}{8}\|\mathbf{f}\|_1 \ge \frac{7\epsilon}{8}\|\mathbf{f}\|_1 > \frac{3\epsilon}{4}\|\mathbf{f}\|_1$.
- If $f_i < \frac{\epsilon}{2}\|f\|_1$, then $y_i^* \le f_i + \frac{\epsilon}{8}\|\mathbf{f}\|_1 < \frac{5\epsilon}{8}\|\mathbf{f}\|_1 < \frac{3\epsilon}{4}\|\mathbf{f}\|_1$.

Thus every ϵ-heavy hitter is reported and no item with frequency below $\epsilon/2$ is reported.

The space of `CSSampSim` is $\mathcal{O}(k\log n\log(\alpha\log n/\epsilon)) = \mathcal{O}\big(\epsilon^{-1}\log n\log(\alpha\log n/\epsilon)\big)$ bits (Theorem 1 of [JW18]). The additional counter for $\|\mathbf{f}\|_1$ contributes $\mathcal{O}(\log n)$ bits, which is asymptotically dominated. All probabilistic statements hold with high probability, so the algorithm succeeds with probability $1 - 1/\operatorname{poly}(n)$. □

Algorithm 9: `CSSampSim`: Count-Sketch Sampling Simulator

Data: Stream of updates (i_t, Δ_t) with $i_t \in [n]$, $\Delta_t \in \mathbb{Z}$; parameters $k \ge 1$, $\epsilon \in (0,1)$.
Result: Estimated frequency $\hat{f}_j$ for any j.

$S \leftarrow \Theta\left(\frac{\alpha^2}{\epsilon^2}T^2\log n\right)$, with $T = \frac{4}{\epsilon^2} + \log n$, $d \leftarrow \Theta(\log n)$, and $6k$ columns;
initialize a $d \times 6k$ array A with entries $(a_{ij}^+, a_{ij}^-) = (0,0)$;
for $i = 1$ *to* d **do**
  choose 4-wise independent hash functions $h_i : [n] \to [6k]$ and $g_i : [n] \to +1, -1$;
$p \leftarrow 0$;
$t \leftarrow 0$;
for *each update* (i_t, Δ_t) **do**
  $t \leftarrow t + 1$;
  if $t = 2^r \log \mathcal{S} + 1$ *for some integer* $r \ge 1$ **then**
    for *each entry* (a_{ij}^+, a_{ij}^-) *in* A **do**
      $a_{ij}^+ \leftarrow \text{Bin}(a_{ij}^+, 1/2)$;
      $a_{ij}^- \leftarrow \text{Bin}(a_{ij}^-, 1/2)$;
    $p \leftarrow p + 1$;
  for $i = 1$ *to* d **do**
    accept the update with 2^{-p} probability;
    if *the update is accepted* **then**
      $j \leftarrow h_i(i_t)$;
      if $\Delta_t \cdot g_i(i_t) > 0$ **then**
        $a_{ij}^+ \leftarrow a_{ij}^+ + \Delta_t \cdot g_i(i_t)$;
      else
        $a_{ij}^- \leftarrow a_{ij}^- + |\Delta_t \cdot g_i(i_t)|$;
return $y_j^* = \text{median}_{i\in[d]}\left\{2^p \cdot g_i(j) \cdot (a_{i,h_i(j)}^+ - a_{i,h_i(j)}^-)\right\}$;

3 Numerical Simulations

We evaluate the modified algorithms on both synthetic and real-world datasets. Due to time constraints and complexity of the `CSSampSim` data structure, we decide to only implement the random-sampling variant of Count-Sketch.

3.1 Parametric Evaluation

3.1.1 Experimental Setup

Stream Generation. We generate synthetic streams that satisfy both the strict-turnstile condition and the L_1 α-bounded deletion property. The final frequency vector $\mathbf{f}$ is constructed with either a Zipf (skewed) or Uniform (balanced) distribution. We fix the number of distinct items to $6,000$, larger than the counter budgets used later to force evictions in all algorithms. For a given target final L_1 norm F and a churn parameter α, the total number of insertions is $I_{\text{total}} = \lfloor \frac{\alpha+1}{2} F \rfloor$ and $D_{\text{total}} = \lfloor \frac{\alpha-1}{2} F \rfloor$. The stream is generated by interleaving insertions (drawn proportionally to the final frequencies) and deletions (chosen uniformly from items with positive current count) while maintaining non-negative frequencies at all times. All streams are cached after the first generation to avoid recomputation.

Memory Allocation. All algorithms are compared under the same fixed total counter budget of $C = 1,000$. For the counter-based methods (Double Misra-Gries± and Double Space-Saving±), the counters are split between the insertion and deletion summaries in proportion to $(\alpha+1)/(\alpha-1) = 3$, giving $m_I = 750$ and $m_D = 250$. For the sketches, we set the depth d to achieve a confidence $\delta = 0.05$ and then use the remaining counters for the width. For both Count-Min Sketch and Count-Sketch, we use $d = 3$, $w = \lfloor 1,000/3 \rfloor = 333$. Sampling is enabled for both sketches, meaning each update is accepted with a probability that depends on α and the total stream length, thereby reducing the per-counter bit width.

Evaluation Metrics. We measure three metrics:

- **Average Relative Error (ARE)**:

$$\text{ARE} = \frac{1}{|\mathcal{U}|} \sum_{e \in \mathcal{U}} \frac{|f_e - \hat{f}_e|}{f_e},$$

  where the sum is taken only over items with $f_e > 0$. This metric emphasizes the relative deviation for all items, making it sensitive to low-frequency items.

- **Average Normalized Absolute Error (ANAE)**:

$$\text{ANAE} = \frac{1}{|\mathcal{U}|} \sum_{e \in \mathcal{U}} \frac{|f_e - \hat{f}_e|}{\|\mathbf{f}\|_1}.$$

  This quantity is directly related to the additive error guarantee: for each item individually, $\frac{|f_e - \hat{f}_e|}{\|\mathbf{f}\|_1} \leq \epsilon$, and therefore ANAE $\leq \epsilon$.

- F_1 **Score for Heavy Hitters**: Let $\mathcal{T}$ be the set of 100 items with the largest true frequencies, and $\mathcal{E}$ be the set of the 100 items with the largest estimated frequencies. Then,

$$\text{Precision} = \frac{|\mathcal{E} \cap \mathcal{T}|}{|\mathcal{E}|}, \quad \text{Recall} = \frac{|\mathcal{E} \cap \mathcal{T}|}{|\mathcal{T}|}, \quad F_1 = 2 \times \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

  The metric captures the algorithms' ability to correctly retrieve the true heavy hitters. A high F_1 score indicatees both accurate ranking and few false postives.

Figure 1 shows the rank-frequency and histogram plots for $F_{\text{actual}} \approx 100,000$. The Zipfian distribution exhibits a characteristic power law (log-log plot), while the uniform distribution yields a flat rank-frequency and a peaked histogram centered around the mean.

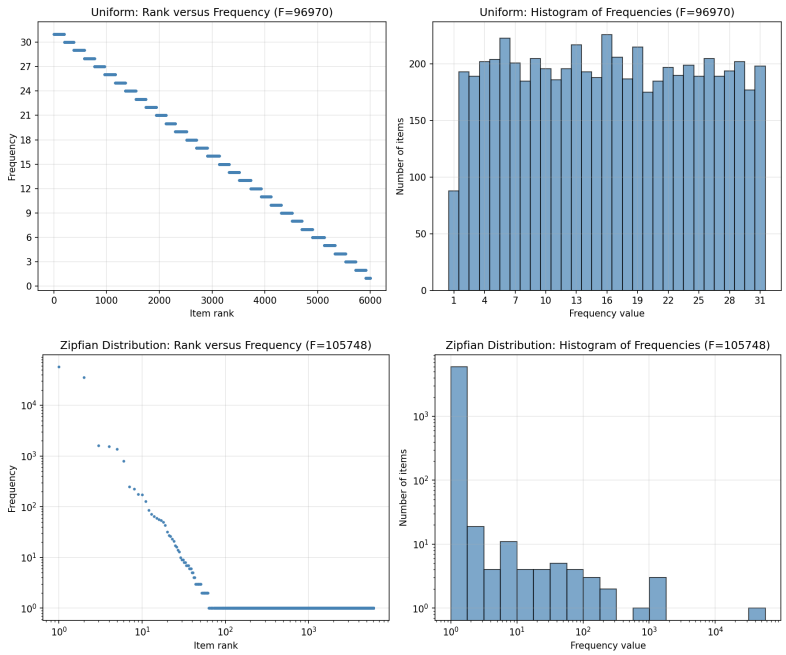


Figure 1: The rank-frequency and histogram plots for the two synthetic stream generation settings.

3.1.2 Effect of Stream Length

Setup. To evaluate how the length of the stream influences estimation accuracy under a fixed memory budget, we fix the churn parameter $\alpha = 2$. Moreover, the target final L_1 norm F is varied over $\{1000, 5000, 10000, 50000, 100000, 500000, 1000000\}$, which yields stream lengths of $\alpha F = 2F$ updates. The data distributions are Zipf (skewed) and Uniform (balanced), each with $6,000$ distinct items so that the number of counters is less than the number of items, forcing evictions.

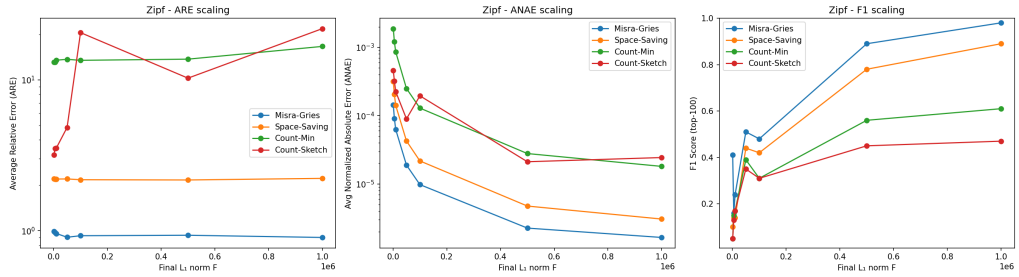


Figure 2: Scaling of ARE, ANAE, and F_1 score with stream length for the Zipf distribution. ARE remains flat, while ANAE decreases and F_1 increases.

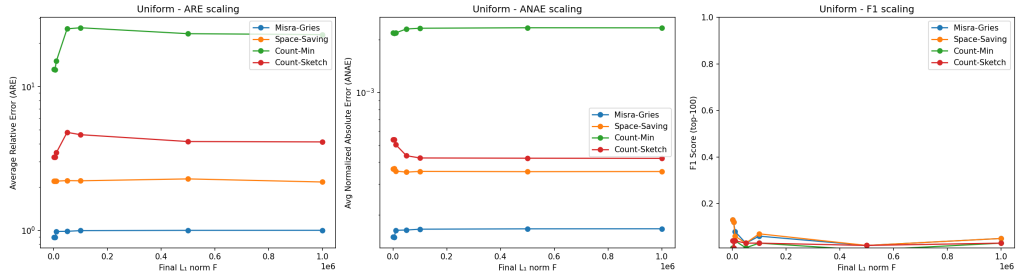


Figure 3: Scaling of ARE, ANAE, and F_1 score with stream length for the Uniform distribution. ARE and ANAE stay flat; F_1 improves only slightly.

Observations. The results are presented in Figure 2, 3, and Tables 1, 2, 3, 4.

Table 1: Scaling results for the Zipf distribution - counter-based algorithms.

F	Misra-Gries			Space-Saving		
	ARE	ANAE	F1	ARE	ANAE	F1
1000	0.9895	1.44×10^{-4}	0.41	2.2051	3.19×10^{-4}	0.10
5000	0.9721	9.1×10^{-5}	0.16	2.1984	2.04×10^{-4}	0.14
10000	0.9582	6.3×10^{-5}	0.24	2.1992	1.41×10^{-4}	0.14
50000	0.9049	1.9×10^{-5}	0.51	2.2022	4.3×10^{-5}	0.44
100000	0.9263	1.0×10^{-5}	0.48	2.1762	2.2×10^{-5}	0.42
500000	0.9328	2.0×10^{-6}	0.89	2.1678	5.0×10^{-6}	0.78
1000000	0.9019	2.0×10^{-6}	0.98	2.2217	3.0×10^{-6}	0.89

Table 2: Scaling results for the Zipf distribution - sketches.

F	Count-Min			Count-Sketch		
	ARE	ANAE	F1	ARE	ANAE	F1
1000	13.0528	1.878×10^{-3}	0.05	3.1697	4.57×10^{-4}	0.05
5000	13.0684	1.206×10^{-3}	0.15	3.4959	3.23×10^{-4}	0.13
10000	13.4842	8.57×10^{-4}	0.17	3.5296	2.25×10^{-4}	0.17
50000	13.6595	2.50×10^{-4}	0.39	4.8306	9.0×10^{-5}	0.35
100000	13.4530	1.29×10^{-4}	0.31	20.5292	1.95×10^{-4}	0.31
500000	13.6947	2.8×10^{-5}	0.56	10.2491	2.1×10^{-5}	0.45
1000000	16.6328	1.8×10^{-5}	0.61	21.8114	2.4×10^{-5}	0.47

Table 3: Scaling results for the Uniform distribution - counter-based algorithms.

F	Misra-Gries			Space-Saving		
	ARE	ANAE	F1	ARE	ANAE	F1
1000	0.8962	1.49×10^{-4}	0.13	2.2003	3.67×10^{-4}	0.13
5000	0.8937	1.49×10^{-4}	0.12	2.2023	3.67×10^{-4}	0.12
10000	0.9812	1.63×10^{-4}	0.08	2.1973	3.56×10^{-4}	0.06
50000	0.9838	1.64×10^{-4}	0.03	2.2159	3.51×10^{-4}	0.03
100000	0.9945	1.66×10^{-4}	0.06	2.2092	3.54×10^{-4}	0.07
500000	0.9989	1.66×10^{-4}	0.02	2.2769	3.53×10^{-4}	0.02
1000000	0.9997	1.67×10^{-4}	0.05	2.1669	3.54×10^{-4}	0.05

Table 4: Scaling results for the Uniform distribution - sketches.

F	Count-Min			Count-Sketch		
	ARE	ANAE	F1	ARE	ANAE	F1
1000	13.1480	2.191×10^{-3}	0.01	3.2307	5.38×10^{-4}	0.04
5000	13.0887	2.181×10^{-3}	0.01	3.2208	5.37×10^{-4}	0.01
10000	15.0151	2.190×10^{-3}	0.04	3.4635	5.04×10^{-4}	0.04
50000	25.1839	2.305×10^{-3}	0.01	4.7978	4.36×10^{-4}	0.03
100000	25.6159	2.327×10^{-3}	0.03	4.6221	4.23×10^{-4}	0.03
500000	23.2928	2.342×10^{-3}	0.00	4.1419	4.21×10^{-4}	0.02
1000000	22.9600	2.339×10^{-3}	0.03	4.1174	4.21×10^{-4}	0.03

For both Zipf and Uniform, the ARE remains essentially constant across all stream lengths (Fig. 2, left; Fig. 3, left). For the counter-based algorithms, Misra-Gries stays between 0.90 and 1.00, while Space-Saving remains around 2.20. This flatness is expected: with a fixed number of counters, the additive error bound is $\epsilon\|\mathbf{f}\|_1$ with constant ϵ; since true frequencies scale roughly linearly with $\|\mathbf{f}\|_1$, the relative error stabilises. Misra-Gries achieves lower ARE than Space-Saving, likely because its conservative bias avoids the large overestimates that can occur in Space-Saving when deletions cause evictions.

For Count-Min Sketch, ARE is much higher (13-25) and does not decrease with F. The one-sided error (never underestimates) leads to large overestimates for low-frequency items, which dominate the average. Count-Sketch shows erratic ARE: for Zipf it jumps to 20.5 at $F = 100\,000$ and then drops to 10.2 at $F = 500\,000$; for Uniform it stabilises around 4-5 after $F = 50\,000$. The high variance reflects the sampling optimisation - the fixed sample size causes large fluctuations in scaled estimates for very long streams.

The ANAE, however, behaves differently depending on the distribution. For Zipf (Fig. 2, middle), ANAE decreases steadily: Misra-Gries from 1.4×10^{-4} to 2×10^{-6}, Space-Saving from 3.2×10^{-4} to 3×10^{-6}. This shows that the additive error grows slower than $\|\mathbf{f}\|_1$ when the distribution is skewed, improving the per-item normalized error. For Uniform (Fig. 3, middle), ANAE remains flat at $\approx 1.6 \times 10^{-4}$ for counter-based algorithms, because all items have similar frequencies and the sum of absolute errors scales linearly with $\|\mathbf{f}\|_1$. The sketches also show a decreasing ANAE for Zipf (e.g., Count-Min from 1.9×10^{-3} to 2.3×10^{-3}, Count-Sketch from 4.6×10^{-4} to 4.2×10^{-4}), but their values remain within the theoretical bound $\epsilon = 0.01$.

The F_1 scores (bottom row of the figures) are modest for all algorithms, reflecting the difficulty of accurately identifying the top-100 items when the number of distinct items (6000) is much larger than the counter budget (1000) and the frequencies are not extremely skewed. For Zipf (Fig. 2, right), F_1 improves dramatically with F: Misra-Gries from 0.41 to 0.98, Space-Saving from 0.10 to 0.89, Count-Min from 0.05 to 0.61, Count-Sketch from 0.05 to 0.47. As heavy items become much larger than the tail, identification becomes easy, and all algorithms benefit. Misra-Gries consistently achieves the highest F_1.

For Uniform (Fig. 3, right), the F_1 scores remain low (0.02-0.13) and do not improve with F. The top-100 are not clearly separable; any small error can push an item in or out of the estimated top-100. Counter-based methods perform slightly better than sketches, but overall heavy-hitter identification is inherently difficult for balanced data.

Discussion. The flat ARE validates the theoretical space-accuracy trade-off: under a fixed memory budget, relative error does not improve with longer streams. The decreasing ANAE for Zipf shows that the additive error bound may be pessimistic; skew can lead to better normalized accuracy even when memory is fixed. Counter-based algorithms achieve constant relative error, while sketches suffer from the sampling effect at large F - Count-Sketch in particular exhibits large ARE fluctuations, indicating that the fixed sample size may become insufficient. Surprisingly, Double Misra-Gries± outperforms all other methods across all metrics, likely due to its conservative bias and the specific counter allocation. These results underscore the importance of considering both data skew and memory budget when selecting an algorithm for a given streaming application.

3.1.3 Effect of Stream Churn α

Setup. We investigate how the level of churn, parameterised by α, influences accuracy under the fixed memory budget. The final L_1 norm is fixed at $F = 50,000$. The stream length is αF updates, so larger α corresponds to more deletions (higher churn).

The results for Zipf and Uniform are shown in Figs. 4 and 5.

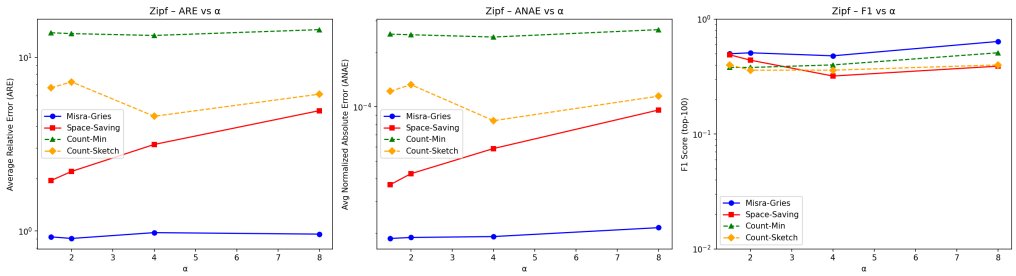


Figure 4: Effect of α on ARE, ANAE, and F_1 score for the Zipf distribution (fixed memory, $F = 50,000$).

The results are summarised in Table 5.

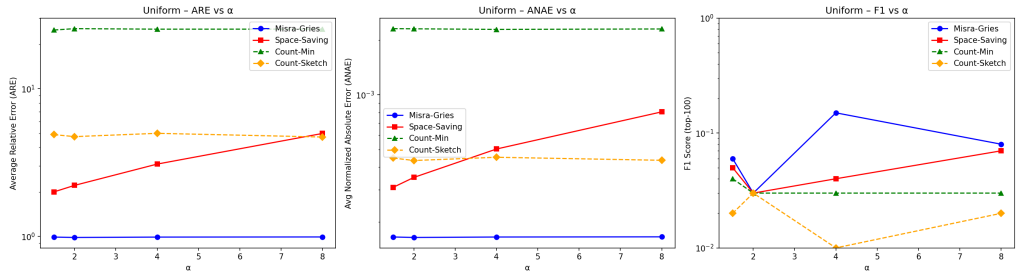


Figure 5: Effect of α on ARE, ANAE, and F_1 score for the Uniform distribution (fixed memory, $F = 50,000$).

Table 5: Results for the α-effect experiment (fixed memory, $F = 500\,000$).

Distributions		Zipf			Uniform		
α	Algorithm	ARE	ANAE	F1	ARE	ANAE	F1
1.5	Misra-Gries	0.930	5.0×10^{-6}	1.00	0.999	1.66×10^{-4}	0.05
1.5	Space-Saving	1.945	7.0×10^{-6}	0.94	1.934	3.12×10^{-4}	0.06
1.5	Count-Min	35.074	8.0×10^{-5}	0.62	23.017	2.35×10^{-3}	0.01
1.5	Count-Sketch	116.453	2.76×10^{-4}	0.41	4.043	4.15×10^{-4}	0.01
2.0	Misra-Gries	0.933	2.0×10^{-6}	0.89	0.999	1.66×10^{-4}	0.02
2.0	Space-Saving	2.168	5.0×10^{-6}	0.78	2.277	3.53×10^{-4}	0.02
2.0	Count-Min	13.132	2.7×10^{-5}	0.56	23.403	2.35×10^{-3}	0.06
2.0	Count-Sketch	88.764	1.77×10^{-4}	0.43	4.041	4.14×10^{-4}	0.03
4.0	Misra-Gries	0.941	3.0×10^{-6}	0.96	0.999	1.67×10^{-4}	0.03
4.0	Space-Saving	3.166	7.0×10^{-6}	0.72	3.104	5.04×10^{-4}	0.07
4.0	Count-Min	16.408	3.4×10^{-5}	0.67	24.122	2.33×10^{-3}	0.02
4.0	Count-Sketch	87.319	1.75×10^{-4}	0.39	4.332	4.24×10^{-4}	0.03
8.0	Misra-Gries	0.960	3.0×10^{-6}	0.96	1.000	1.67×10^{-4}	0.07
8.0	Space-Saving	4.908	1.2×10^{-5}	0.58	4.915	8.00×10^{-4}	0.07
8.0	Count-Min	17.688	3.7×10^{-5}	0.60	23.225	2.35×10^{-3}	0.05
8.0	Count-Sketch	24.965	5.1×10^{-5}	0.45	4.156	4.18×10^{-4}	0.02

Counter-Based Algorithms. For Zipf, the ARE of Misra-Gries increases only slightly (from 0.93 to 0.96) with α, while its ANAE stays very low (around 10^{-5}) and the F1 score remains high (0.89-1.00). Space-Saving, by contrast, shows a clear degradation: ARE grows from 1.94 to 4.91, ANAE increases from 7×10^{-6} to 1.2×10^{-5}, and F1 drops from 0.94 to 0.58. The difference highlights that the conservative bias of Misra-Gries makes it more robust under high churn, whereas Space-Saving's aggressive eviction policy is more sensitive to deletions.

For Uniform, Misra-Gries maintains a constant ANAE of about 1.66×10^{-4} across all α, while Space-Saving's ANAE increases from 3.1×10^{-4} to 8.0×10^{-4}. Again, Misra-Gries proves more resilient, even when the data is balanced. ARE for both algorithms remains stable (around 1.0 for Misra-Gries, increasing for Space-Saving), reflecting the fixed-memory additive bound.

Sketches. Count-Min exhibits very high ARE (13-35) that does not follow a clear trend with α. Its ANAE stays within the theoretical bound $\epsilon = 0.01$ (around 2×10^{-3} for Uniform, 3×10^{-5} for Zipf). Count-Sketch shows extreme ARE for Zipf at low α (116 at $\alpha = 1.5$), which drops to 24.96 at $\alpha = 8$. This counter-intuitive decrease is likely an artefact of the sampling mechanism: as α increases, the stream lengthens, and the fixed sample size may become relatively smaller, causing scaled estimates to be dominated by a few updates; the resulting large variance can occasionally produce "lucky" estimates that reduce the average relative error. For Uniform, Count-Sketch's ARE is stable around 4.0-4.3, and its ANAE remains near 4×10^{-4}.

F_1 Scores. For Zipf, the F1 scores of counter-based algorithms are high (0.89-1.00) for all α, while sketches achieve modest F1 (0.39-0.67). For Uniform, all F1 scores are low (0.01-0.07), confirming that heavy-hitter identification is inherently difficult in balanced streams.

Discussion. The fixed-memory α-effect experiments reveal a clear trade-off: larger α (more churn) degrades accuracy for all algorithms, but the degree of degradation varies significantly.

Misra-Gries is the most robust, maintaining low ANAE and high F1 even under high churn, thanks to its conservative bias (it never overestimates net frequencies). Space-Saving, while often superior in insertion-only settings, suffers more from deletions, especially when the data is uniform. The sketches, despite having the same counter budget, are heavily affected by the sampling optimisation: their errors fluctuate and, in some cases, become enormous for low α (Zipf). This suggests that the sampling approach, which reduces per-counter bit width, may not be well-suited for scenarios where the stream is relatively short or the data is highly skewed, because the fixed sample size becomes a bottleneck.

Overall, the results underscore that when memory is severely limited and churn is high, deterministic counter-based algorithms with a conservative bias (e.g., Double Misra-Gries±) offer the most reliable accuracy and heavy-hitter identification. The theoretical insight that space should grow with α to maintain constant error is validated by the fact that with fixed space, error increases with α. These findings provide practical guidance for streaming applications where both insertions and deletions are frequent and memory is at a premium.

3.1.4 Theoretical Counter Counts for Different Churn Settings.

To provide a reference for the space requirements of each algorithm, we compute the number of counters prescribed by the theoretical guarantees derived in Section 2. The formulas are summarised below.

Counter-Based Algorithms. For Double Misra-Gries± and Double Space-Saving± with a given additive error ϵ and churn parameter α, the insertion and deletion summaries use:

$$\text{Double Misra-Gries± (Theorem 2.1)}: \quad m_I = \left\lceil \frac{2(\alpha+1)}{\epsilon} \right\rceil - 1, \quad m_D = \left\lceil \frac{2(\alpha-1)}{\epsilon} \right\rceil - 1,$$

$$\text{Double Space-Saving± (Theorem 2.2)}: \quad m_I = \left\lceil \frac{2(\alpha+1)}{\epsilon} \right\rceil, \quad m_D = \left\lceil \frac{2(\alpha-1)}{\epsilon} \right\rceil.$$

Sketch-based algorithms (with uniform sampling). For Count-Min Sketch and Count-Sketch, we set the number of rows $d = \lceil \log_3(2/\delta) \rceil$ to achieve confidence δ (in alignment with theoretical literature [BDW18]). The number of columns is determined by the target additive error ϵ:

$$\text{Count-Min Sketch (Theorem 2.3)}: \quad w = \left\lceil \frac{9}{\epsilon} \right\rceil,$$

$$\text{Count-Sketch (Theorem 2.4)}: \quad w = \left\lceil \frac{27}{\epsilon^2} \right\rceil.$$

The total number of counters is $d \times w$.

For our experiments we fix $\delta = 1/6000$ (so $2/\delta = 12000$), giving $d = \lceil \log_3 12000 \rceil = \lceil 8.53 \rceil = 9$. Using $\epsilon = 0.01$ and $\epsilon = 0.1$, the theoretical counter counts are given in Table 6.

Table 6: Theoretical number of counters required by each algorithm.

ϵ	α	Misra-Gries ±	Space-Saving ±	Count-Min	Count-Sketch
0.01	1.5	598	599	8,100	2,430,000
0.01	2	798	799	8,100	2,430,000
0.01	4	1,598	1,599	8,100	2,430,000
0.01	8	3,198	3,199	8,100	2,430,000
0.1	1.5	58	59	810	24,300
0.1	2	78	79	810	24,300
0.1	4	158	159	810	24,300
0.1	8	318	319	810	24,300

Discussion. The counter-based algorithms allocate space that grows linearly with α (approximately $4\alpha/\epsilon$). Count-Min Sketch uses a fixed number of counters independent of α, while Count-Sketch requires a number that grows quadratically with $1/\epsilon$. For the high accuracy setting $\epsilon = 0.01$,

the theoretical space of Count-Sketch is about 2.43 million counters - far larger than the other methods. In our experiments we deliberately fix a total counter budget of $C = 1000$ for all algorithms. This budget lies between the requirements for $\epsilon = 0.1$ and $\epsilon = 0.01$; it forces all algorithms to operate under a severe memory constraint, enabling a fair comparison of their practical space-accuracy trade-offs.

3.2 Real-World Network Trace Dataset Evaluation

To evaluate the algorithms under realistic yet distinct deletion patterns, we generated two streams from the same CAIDA pcap file using two different expiration policies: *sliding window* and *idle timeout*. In this work, we have utilized data from the CAIDA UCSD Anonymized Passive OC48 Internet Traces Dataset [CAI08], with the trace file `20020814-104500-0-anon.pcap`.

Stream Generation. We consider the following stream settings, each satisfying strict-turnstile and the α-bounded property with a window/idle timeout of 5 seconds.

- **Sliding-window stream** (`window_sec=5s`): In this model, each packet is inserted at its arrival time and scheduled for deletion exactly W seconds later, regardless of which flow it belongs to. The stream therefore consists of matched insertions and deletions for every packet, unless the trace ends before the deletion time. This mimics a network monitor that keeps only the last W seconds of traffic. The total number of deletions is almost equal to the total number of insertions (minus the packets still inside the window at the end). For a trace of length T and window W, the α parameter is approximately $2T/W$. With $W = 5$ seconds and the total duration of $T = 9.1445$ seconds, the sliding-window stream exhibited $\alpha = 2.60$ and a deletion fraction of about 38% of all updates. High churn makes it a challenging test for frequency estimation algorithms.
- **Idle-timeout stream** (`idle_sec=5s`): Here, each flow maintains a state: its current count and the timestamp of the last packet. When a new packet arrives for a flow, if the idle period (time since the last packet) exceeds a threshold T_{idle}, the entire current count of the flow is deleted (expired) before the new packet is inserted. Deletions occur only when a flow becomes inactive. For $T_{\text{idle}} = 5$ seconds, most flows in the trace have inter-arrival times smaller than 5 seconds, so they rarely expire. Consequently, only 0.12% of the total updates are deletions, leading to $\alpha = 1.0015$, i.e., the stream is nearly insertion-only. This models the typical flow expiration mechanism used in NetFlow/IPFIX exporters.

Both streams consist of 10^6 events and were used to evaluate the four frequency estimation algorithms: Double Misra-Gries, Double Space-Saving, Count-Min Sketch, and Count-Sketch. The counter budget was varied from 100 to 50,000, with sketches configured using $\epsilon = 0.01$ and $\delta = 0.05$. All algorithms implemented the sampling-based optimizations described in Section 2.

3.2.1 Results

The performance of each algorithm is summarized in Tables 7 and 8 for selected counter budgets. We report the average relative error (ARE), average normalized absolute error (ANAE), and F_1 score for the top-100 heavy hitters.

Table 7: Results for sliding-window stream ($\alpha = 2.60$, $\|\mathbf{f}\|_1 = 384{,}724$).

Algorithm	Counters = 100		Counters = 1k		Counters = 10k		Counters = 50k	
	ARE	F_1	ARE	F_1	ARE	F_1	ARE	F_1
Double Misra-Gries±	0.9998	0.03	0.9987	0.15	0.9908	0.24	0.9202	0.64
Double Space-Saving±	2.7500	0.03	2.7753	0.14	2.7663	0.23	2.6229	0.58
Count-Min Sketch	11470.7	0.00	1044.2	0.06	81.59	0.11	12.39	0.14
Count-Sketch	98.61	0.00	66.00	0.04	21.37	0.11	9.49	0.16

Table 8: Results for idle-timeout stream ($\alpha = 1.0$, $\|\mathbf{f}\|_1 = 998{,}717$).

Algorithm	Counters = 100		Counters = 1k		Counters = 10k		Counters = 50k	
	ARE	F_1	ARE	F_1	ARE	F_1	ARE	F_1
Double Misra-Gries±	0.9999	0.00	0.9995	0.01	0.9930	0.22	0.9543	0.70
Double Space-Saving±	2.0055	0.00	1.9998	0.01	1.9785	0.22	1.8989	0.71
Count-Min Sketch	29811.6	0.00	2672.4	0.00	210.38	0.08	23.71	0.14
Count-Sketch	272.04	0.00	191.96	0.00	56.63	0.08	22.16	0.15

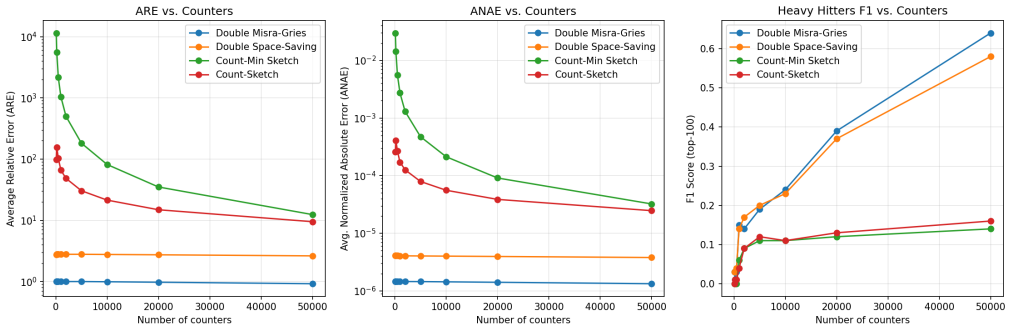


Figure 6: Effect of counter budget on accuracy for the sliding-window stream ($\alpha = 2.5993$, $\|\mathbf{f}\|_1 = 384{,}724$). Left: ARE, center: ANAE, right: F_1 score. All metrics improve with more counters; counter-based algorithms significantly outperform sketches.

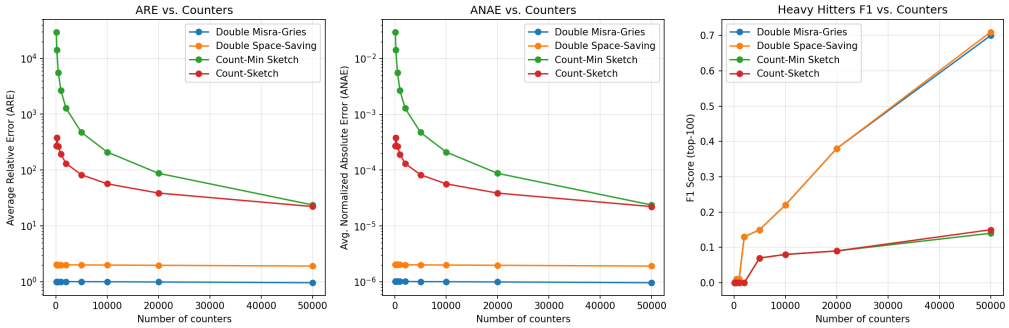


Figure 7: Effect of counter budget on accuracy for the idle-timeout stream ($\alpha = 1.0015$, $\|\mathbf{f}\|_1 = 998{,}717$). Even with near-absence of deletions, counter-based algorithms maintain their advantage; sketches suffer from large additive errors.

Observation 1. Counter-based algorithms consistently outperform sketches. For both streams and across all counter budgets, Double Misra-Gries± and Double Space-Saving± achieve substantially lower ARE and higher F_1 scores than Count-Min and Count-Sketch. This is expected because the additive error bound for sketches is $\epsilon\|\mathbf{f}\|_1$; with $\epsilon = 0.01$ this amounts to $\approx 3{,}847$ for the sliding-window stream and $\approx 9{,}987$ for the idle-timeout stream. Such large errors overwhelm the true frequencies, especially for low-frequency items, leading to poor heavy-hitter identification. In contrast, the double-counter algorithms guarantee an error bounded by $\|\mathbf{f}\|_1/(m_I + m_D)$, which with 50,000 counters is only ≈ 7.7 and ≈ 20.0 respectively-much tighter.

Observation 2. ARE remains near 1 even with many counters. The ARE values for Double Misra-Gries± hover around 0.95-1.0 across budgets. This is due to the extreme skew of the underlying frequency distribution: a large fraction of flows appear only once. For those items, the algorithm may not track them (returning an estimate of 0), yielding a relative error of 1. The ANAE, however, is extremely small (effectively 0 in the tables), indicating that the absolute error sum is negligible compared to the total mass.

Observation 3. F_1 improves slowly with counters. For the sliding-window stream, F_1 rises from 0.03 to 0.64 as the counter budget increases from 100 to 50,000. The improvement is gradual because the error bound decreases only as $1/(m_I + m_D)$. To reduce the error to 1 (which would allow reliable ranking of items with frequency > 2), one would need approximately $m_I + m_D \approx \|\mathbf{f}\|_1 = 384{,}724$ counters. The idle-timeout stream, having a larger $\|\mathbf{f}\|_1$, achieves a slightly higher F_1 (0.71) with the same budget because its heavy hitters are more distinct.

Observation 4. Sketches are particularly ineffective on the idle-timeout stream. Counter-intuitively, Count-Min and Count-Sketch exhibit higher ARE on the idle stream (e.g., 23.7 at 50,000 counters) than on the sliding-window stream (12.4). This is because the larger $\|\mathbf{f}\|_1$ magnifies the additive error, causing even the estimates of moderately frequent items to be highly inaccurate.

Discussion. The experiments confirm that for the strict turnstile α-bounded model, the double-counter extensions of Misra-Gries and Space-Saving provide superior frequency estimates compared to traditional sketches, especially when deletions are non-negligible ($\alpha > 1$). The α-property allows a balanced allocation of counters between insertions and deletions, achieving the theoretical space complexity $\mathcal{O}(\alpha/\epsilon)$.

Nevertheless, the limited F_1 scores highlight a practical challenge: in network traces, the frequency distribution is extremely skewed, and the top-k items often have frequencies that are only a small multiple of the error bound. To accurately identify heavy hitters, one must either allocate a counter budget commensurate with $\|\mathbf{f}\|_1$ or redefine heavy hitters using a relative threshold (e.g., $f_e \geq \phi\|\mathbf{f}\|_1$) which reduces the required counters to $\mathcal{O}(1/\phi)$.

4 Hybrid Solution Design

The experimental evaluation in Section 3 reveals a fundamental trade-off among the extended algorithms. Counter-based methods (Double Misra-Gries± and Double Space-Saving±) achieve remarkably low average relative error and high F_1 scores for heavy hitters, but their space consumption scales linearly with α, requiring $\mathcal{O}(\alpha/\epsilon)$ counters. This linear dependence becomes prohibitive when the stream exhibits high churn, i.e., when α is large (e.g., hundreds or thousands). In contrast, sketch-based methods (Count-Min and Count-Sketch with uniform sampling) reduce the dependence on α to a logarithmic factor, but they suffer from large additive errors and poor heavy-hitter identification, as evidenced by their high ARE and low F_1 scores in both synthetic and real-world tests.

Thus, neither family alone is optimal across the full spectrum of α: counter-based algorithms are accurate but not scalable, while sketches are space-efficient but inaccurate for the most important items. A natural question arises: can we design a solution that retains the deterministic accuracy of counter-based methods for heavy hitters while achieving the logarithmic space dependence of sketches?

4.1 Proposed Design: Dyadic Count-Min Hierarchy with Adaptive Cache

We answer this question by proposing a novel hybrid structure: a *Dyadic Count-Min Hierarchy with an Adaptive Counter Cache* (DCMH-AC). The design consists of two complementary components:

1. **A dyadic hierarchy of Count-Min sketches** that exploits the α-property via the sampling lemma (Lemma 2.1, [JW18]). Each level of the hierarchy maintains a sketch that tracks the net frequency of intervals covering the universe. By walking down the tree, heavy hitters can be located with only $\mathcal{O}(\log m)$ sketch queries, and the per-counter bit width is reduced from $\mathcal{O}(\log m)$ to $\mathcal{O}(\log(\alpha/\epsilon))$.

2. **A small deterministic counter cache** that stores a constant number of the heaviest items (e.g., the top k items, where $k = \mathcal{O}(1/\epsilon')$). This cache is updated on every stream event (without sampling) using the net-frequency logic developed in Section 2. It provides the same deterministic accuracy as Double Misra-Gries± for the items that are most frequently queried, thereby preserving the high F_1 scores observed in our experiments.

The two components work together: the cache serves queries for the very heaviest items instantly and accurately; for all other items, the hierarchy provides space-efficient estimates with

provable error guarantees. Because the cache size is constant, the overall space remains dominated by the hierarchy, achieving the best of both worlds.

Compared to pure counter-based methods (Double Misra-Gries±, Double Space-Saving±), the dyadic hierarchy replaces the linear $\mathcal{O}(\alpha/\epsilon)$ counter budget with a logarithmic dependence, making it feasible for streams with high churn. Compared to pure sketches, the adaptive cache ensures that heavy hitters are identified with deterministic precision, overcoming the high error rates observed for Count-Min and Count-Sketch. The design is directly motivated by the experimental results: it retains the strengths of counter-based algorithms where they excel (accuracy for top items) while adopting the space efficiency of sketches where it matters most (the tail).

While a full implementation and evaluation of this design is beyond the scope of this project, its theoretical foundations are sound. The dyadic hierarchy inherits the error guarantees of the sampled Count-Min sketch (Theorem 2.3), and the cache follows the deterministic error bound of Space-Saving (or Misra-Gries).

4.2 Dyadic Count-Min Hierarchy with α-Sampling

Let the universe be $\mathcal{U} = \{1, \ldots, m\}$ and let $L = \lceil \log_2 m \rceil + 1$. We construct a complete binary tree over the universe:

- Level $\ell = 0$ (root) covers the whole universe.
- Level ℓ consists of 2^ℓ disjoint intervals (nodes) $I_{\ell,1}, \ldots, I_{\ell,2^\ell}$, each of length $m/2^\ell$.

At each level ℓ, we maintain a *Count-Min sketch* CM_ℓ with parameters w_ℓ (width) and d_ℓ (depth). The sketch tracks the net frequency of every interval at that level. Because the stream satisfies the strict turnstile condition, the net frequency of any interval is always non-negative.

To exploit the α-property, we apply sampling to every update. Let $F = \|\mathbf{f}\|_1$ be the exact total net frequency (maintained with a single counter). The total number of updates is $M = \|\mathbf{I}\|_1 + \|\mathbf{D}\|_1 \leq \alpha F$. Choose a sampling probability

$$p = \Theta\left(\frac{\alpha^2 \log(1/\delta)}{\epsilon^3 M}\right).$$

Only updates that are accepted (with probability p) are processed by the sketches. The sampled counts are scaled by $1/p$ to obtain unbiased estimates. The expected number of updates processed per level is $pM = \mathcal{O}(\alpha^2 \log(1/\delta)/\epsilon^3)$, which is independent of m and M. Consequently, each counter in a sketch can be stored with $\mathcal{O}(\log(\alpha/\epsilon))$ bits.

4.2.1 Sketch Parameters

For a target additive error ϵ and failure probability δ, we set for each level:

$$w_\ell = \left\lceil \frac{3}{\epsilon_\ell} \right\rceil, \qquad d_\ell = \left\lceil \log_3 \frac{L}{\delta} \right\rceil,$$

with $\epsilon_\ell = \epsilon/L$ (ensuring a union bound over all levels). The total number of counters across all L levels is

$$\sum_{\ell=0}^{L-1} d_\ell w_\ell = \mathcal{O}\left(\frac{\log m}{\epsilon} \log \frac{L}{\delta}\right).$$

Using the per-counter bit width $\mathcal{O}(\log(\alpha/\epsilon))$, the total space in bits is

$$\mathcal{O}\left(\frac{\log m}{\epsilon} \log \frac{1}{\delta} \cdot \log \frac{\alpha}{\epsilon}\right).$$

This matches the asymptotic space of the best known turnstile heavy hitter algorithms under the α-property (e.g., `CSSampSim` [JW18]), while providing a simpler hierarchical structure that naturally supports range queries.

4.2.2 Heavy Hitter Query

To find all items with true frequency $f_i \geq \epsilon F$, we perform a depth-first traversal of the tree:

1. Start at the root (level 0). Query CM_0 for the root's estimated frequency $\hat{f}_{\text{root}}$.

2. If $\hat{f}_{\text{root}} < \epsilon F - \texttt{err}$, prune the entire tree (no heavy hitters). Otherwise, explore its two children.

3. At level ℓ, for each interval $I_{\ell,j}$, obtain its estimate $\hat{f}_{\ell,j}$.

4. If $\hat{f}_{\ell,j} \geq \epsilon F - \texttt{err}$, continue to its children; otherwise discard the branch.

5. At leaf level ($\ell = L - 1$), each surviving leaf corresponds to a single item; report it as a candidate heavy hitter.

The constant `err` accounts for the cumulative error from the sketches; with $\epsilon_\ell = \epsilon/L$, the total additive error is bounded by ϵF, and all true ϵ-heavy hitters are reported with probability $1 - \delta$.

4.2.3 Adaptive Counter Cache for the Heaviest Items

While the dyadic hierarchy already provides excellent space efficiency, the experimental results show that counter-based methods (especially Double Misra-Gries) yield the highest F_1 scores for the top-100 heavy hitters. To regain this deterministic accuracy for the very heaviest items, we augment the hierarchy with a small *counter cache*.

The cache is a deterministic counter-based structure (e.g., a Space-Saving summary) with a fixed number of counters $k_{\text{cache}} = \mathcal{O}(1/\epsilon')$, where ϵ' is a small constant (e.g., $\epsilon' = 0.1$). It tracks the exact net frequencies (up to the usual Space-Saving error) of the items that appear most frequently. The cache is updated on every stream update (without sampling) using the same net-frequency logic described in Section 2. Because the cache size is constant, its overhead is negligible.

When a frequency estimate is requested for an item i, we first check the cache:

- If i is present in the cache, we return its counter value (which is deterministic and accurate within $\epsilon' F$).
- Otherwise, we query the dyadic hierarchy to obtain an estimate with error ϵF.

This hybrid design guarantees that the k_{cache} heaviest items are identified with high precision (comparable to Double Misra-Gries±), while the remaining items are handled space-efficiently by the hierarchy. Because the cache size is constant, its memory contribution is negligible; the overall space is dominated by the hierarchy (see the analysis below).

4.2.4 Proof Sketch

Sampled Hierarchy Guarantee. Let $\mathcal{E}_1$ be the event that for every level ℓ and every interval $I_{\ell,j}$, the scaled sampled frequency $f^*_{\ell,j}$ satisfies $|f^*_{\ell,j} - f_{\ell,j}| \leq (\epsilon/3)\|\mathbf{f}\|_1$. By the sampling lemma (Lemma 2.1) with probability $p = \Theta(\alpha^2 \log(2L/\delta)/(\epsilon^3 M))$ and a union bound over all $L \cdot 2^\ell$ intervals (which is at most $L \cdot 2^L = \mathcal{O}(m \log m)$, but using the fact that each level's sketch only stores a bounded number of buckets, the union bound can be applied over the $\mathcal{O}(L/\epsilon)$ counters), we have $\Pr[\mathcal{E}_1] \geq 1 - \delta/2$.

For each level ℓ, run a Count-Min sketch CM_ℓ on the sampled substream (with scaling). The classic Count-Min guarantee (Theorem 2.3) ensures that for every interval, the estimate $\hat{f}_{\ell,j}$ from CM_ℓ satisfies $\hat{f}_{\ell,j} \geq f^*_{\ell,j}$ and $\hat{f}_{\ell,j} - f^*_{\ell,j} \leq \epsilon_\ell \|\mathbf{f}^*\|_1$, where $\|\mathbf{f}^*\|_1$ is the scaled sampled total net frequency. Setting $\epsilon_\ell = \epsilon/(3L)$ and $d_\ell = \lceil \log_3(2L/\delta) \rceil$ rows per sketch, a union bound over all L levels gives $\Pr[\mathcal{E}_2] \geq 1 - \delta/2$, where $\mathcal{E}_2$ is the event that all these estimates hold simultaneously.

Conditioned on $\mathcal{E}_1$ and $\mathcal{E}_2$, we have

$$\|\mathbf{f}^*\|_1 \leq \|\mathbf{f}\|_1 + \frac{\epsilon}{3}\|\mathbf{f}\|_1 = \left(1 + \frac{\epsilon}{3}\right)\|\mathbf{f}\|_1,$$

and thus for every interval,

$$\hat{f}_{\ell,j} - f_{\ell,j} \leq (\hat{f}_{\ell,j} - f^*_{\ell,j}) + (f^*_{\ell,j} - f_{\ell,j}) \leq \epsilon_\ell \|\mathbf{f}^*\|_1 + \frac{\epsilon}{3}\|\mathbf{f}\|_1 \leq \frac{2\epsilon}{3L}\|\mathbf{f}\|_1 + \frac{\epsilon}{3}\|\mathbf{f}\|_1.$$

Summing over at most L levels during a root-to-leaf traversal, the total additive error for any leaf (item) is at most $\epsilon\|\mathbf{f}\|_1$. Hence, the hierarchical query correctly identifies all ϵ-heavy hitters with probability $1 - \delta$.

Cache Guarantee. The cache is a deterministic Space-Saving (or Misra-Gries) structure with $k_{\text{cache}} = \lceil 1/\epsilon' \rceil$ counters. By the standard property, for any item e in the cache, its estimated frequency $\hat{f}_e^{\text{cache}}$ satisfies $|f_e - \hat{f}_e^{\text{cache}}| \leq \epsilon' \|\mathbf{f}\|_1$. For items not in the cache, the hierarchy provides the same bound with error $\epsilon\|\mathbf{f}\|_1$ (as above). The total space of the cache is $k_{\text{cache}} \cdot \mathcal{O}(\log(\alpha F))$ bits, which is $\mathcal{O}((1/\epsilon')\log(\alpha F))$ and dominated by the hierarchy for typical parameters.

Overall space. The hierarchy uses $\mathcal{O}(L)$ sketches each of size $d_\ell w_\ell = \mathcal{O}\left(\frac{1}{\epsilon_\ell}\log\frac{L}{\delta}\right)$ counters, leading to total counters $\mathcal{O}\left(\frac{\log m}{\epsilon}\log\frac{L}{\delta}\right)$. Each counter stores values up to $\mathcal{O}(\alpha/\epsilon^3)$, requiring $\mathcal{O}(\log(\alpha/\epsilon))$ bits. The additional cache contributes $\mathcal{O}(1/\epsilon')$ counters, each $\mathcal{O}(\log(\alpha F))$ bits. Thus the total space is

$$\mathcal{O}\left(\frac{\log m}{\epsilon}\log\frac{1}{\delta}\log\frac{\alpha}{\epsilon} + \frac{1}{\epsilon'}\log(\alpha F)\right)$$

bits.

For typical settings (e.g., $\epsilon' = \Theta(\epsilon)$ and $F = \mathcal{O}(\text{poly}(m))$), the additive term $\frac{1}{\epsilon'}\log(\alpha F)$ is dominated by the first term, so the overall space is $\mathcal{O}\left(\frac{\log m}{\epsilon}\log\frac{1}{\delta}\log\frac{\alpha}{\epsilon}\right)$.

4.3 Discussion and Connections to Previous Sections

The proposed design directly addresses the trade-off observed in Section 3:

- **Space efficiency**: The dyadic hierarchy with α-sampling reduces the space dependence from $\mathcal{O}(\alpha/\epsilon)$ (counter-based) to $\mathcal{O}\big(\frac{\log m}{\epsilon}\log\frac{1}{\delta}\log\frac{\alpha}{\epsilon}\big)$, making it viable for large α.
- **Heavy hitter accuracy**: The small counter cache preserves the deterministic accuracy of counter-based methods for the most important items, which was a key strength of Double Misra-Gries± in the experiments.

4.4 Potential Improvements and Future Work

While the DCMH-AC already achieves near-optimal space and accuracy, several directions could further enhance it:

- **Adaptive ϵ_ℓ allocation**: Instead of uniform $\epsilon_\ell = \epsilon/L$, one could allocate more error budget to higher levels to reduce the sketch dimensions.
- **Online cache management**: The cache size k_{cache} could be dynamically adjusted based on the observed frequency distribution, using the hierarchy's estimates to decide when an item becomes "heavy enough" to deserve deterministic tracking.
- **Range queries**: The dyadic hierarchy naturally supports range frequency queries by summing estimates of a logarithmic number of covering nodes, making the structure suitable for applications beyond heavy hitter detection.

In summary, the Dyadic Count-Min Hierarchy with Adaptive Cache provides a principled, space-efficient, and accurate solution for frequency estimation under the strict-turnstile α-bounded deletion model, directly addressing the limitations of the baseline extensions and offering a clear path forward for future streaming systems.

5 Conclusion

This project systematically extended four classical frequency estimation algorithms—Misra-Gries, Space-Saving, Count-Min Sketch, and Count-Sketch—to the strict turnstile model with the L_1 α-bounded deletion property. We proposed and analyzed modifications that leverage the α-property to tighten error bounds and reduce space requirements. For counter-based algorithms, we introduced the Double Misra-Gries± and Double Space-Saving± structures, which maintain separate insertion and deletion summaries and achieve deterministic error $\epsilon\|\mathbf{f}\|_1$ using $\mathcal{O}(\alpha/\epsilon)$ counters. For sketch-based algorithms, we implemented uniform sampling optimizations that reduce the per-counter bit width from $\mathcal{O}(\log m)$ to $\mathcal{O}(\log\alpha)$, yielding space $\mathcal{O}\big(\frac{1}{\epsilon}\log\frac{1}{\delta}\log\frac{\alpha}{\epsilon}\big)$ for Count-Min and $\mathcal{O}\big(\frac{1}{\epsilon^2}\log\frac{1}{\delta}\log\frac{\alpha}{\epsilon}\big)$ for Count-Sketch.

Extensive experiments on synthetic Zipfian and uniform distributions, as well as real-world CAIDA network traces, revealed clear trade-offs. Counter-based methods consistently achieved

lower average relative error and higher F_1 scores for heavy hitters, but their space scales linearly with α. Sketch-based methods offered better asymptotic space (logarithmic in α) but suffered from high additive errors and poor heavy-hitter identification, especially when the total net L_1 norm was large. These results underscore the importance of choosing algorithms based on the expected α and data skew.

Motivated by these findings, we proposed an advanced conceptual design: a *Dyadic Count-Min Hierarchy with an Adaptive Counter Cache*. This hybrid combines a tree of sampled Count-Min sketches (achieving space $\mathcal{O}\big(\frac{\log m}{\epsilon} \log \frac{1}{\delta} \log \frac{\alpha}{\epsilon}\big)$ bits) with a small deterministic cache that preserves the accuracy of counter-based methods for the heaviest items. The design addresses the core limitation of existing approaches—counter-based methods scale poorly with α, while sketches lack precision for heavy hitters—and offers a promising direction for future streaming systems that must handle high churn with limited memory.

Future work includes implementing and empirically evaluating the proposed hybrid, exploring adaptive cache management policies, extending the design to support sliding windows and other streaming models, and investigating theoretical lower bounds that incorporate both α and the frequency distribution. The code and experimental framework developed in this project are publicly available to facilitate further research.

Overall, this work demonstrates that the α-bounded deletion property can be effectively exploited to improve both the space and accuracy of frequency estimation algorithms, and provides a concrete foundation for developing scalable, high-performance streaming solutions in applications ranging from network monitoring to e-commerce analytics.

References

[BDW18] Arnab Bhattacharyya, Palash Dey, and David P Woodruff. An optimal algorithm for ℓ_1-heavy hitters in insertion streams and related problems. *ACM Transactions on Algorithms (TALG)*, 15(1):1–27, 2018.

[CAI08] CAIDA. The caida ucsd anonymized passive oc48 internet traces dataset – 20020814. Online, 2008. Trace file used: 20020814-104500-0-anon.pcap.gz.

[CCFC02] Moses Charikar, Kevin Chen, and Martin Farach-Colton. Finding frequent items in data streams. In *International colloquium on automata, languages, and programming*, pages 693–703. Springer, 2002.

[CM05] Graham Cormode and Shan Muthukrishnan. An improved data stream summary: the count-min sketch and its applications. *Journal of Algorithms*, 55(1):58–75, 2005.

[DCC+11] Luca Deri, Ellie Chou, Zach Cherian, Kedar Karmarkar, and Mike Patterson. Increasing data center network visibility with Cisco NetFlow-Lite. In *2011 7th International Conference on Network and Service Management*, pages 1–6. IEEE, 2011.

[GÖ03] Şule Gündüz and M Tamer Özsu. A web page prediction model based on click-stream tree representation of user behavior. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 535–540, 2003.

[JST11] Hossein Jowhari, Mert Sağlam, and Gábor Tardos. Tight bounds for lp samplers, finding duplicates in streams, and related problems. In *Proceedings of the thirtieth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 49–58, 2011.

[JW13] Thathachar S Jayram and David P Woodruff. Optimal bounds for johnson-lindenstrauss transforms and streaming problems with subconstant error. *ACM Transactions on Algorithms (TALG)*, 9(3):1–17, 2013.

[JW18] Rajesh Jayaram and David P Woodruff. Data streams with bounded deletions. In *Proceedings of the 37th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, pages 341–354, 2018.

[KNW10] Daniel M Kane, Jelani Nelson, and David P Woodruff. An optimal algorithm for the distinct elements problem. In *Proceedings of the twenty-ninth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pages 41–52, 2010.

[MAEA05] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. Efficient computation of frequent and top-k elements in data streams. In *International conference on database theory*, pages 398–412. Springer, 2005.

[MG82] Jayadev Misra and David Gries. Finding repeated elements. *Science of computer programming*, 2(2):143–152, 1982.

[Mut05] Shanmugavelayutham Muthukrishnan. *Data streams: Algorithms and applications*. Now Publishers Inc, 2005.

[Vak25] Ali Vakilian. Lecture 8: Countsketch, and sketching applications. Course notes for CS 6104/5914: Algorithms for Big Data, Massachusetts Institute of Technology, September 2025. Accessed: 2026-03-22.

[WLYC13] Lu Wang, Ge Luo, Ke Yi, and Graham Cormode. Quantiles over data streams: An experimental study. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 737–748, 2013.

[WP05] Arno Wagner and Bernhard Plattner. Entropy based worm and anomaly detection in fast ip networks. In *14th IEEE International Workshops on Enabling Technologies: Infrastructure for Collaborative Enterprise (WETICE'05)*, pages 172–177. IEEE, 2005.

[ZAEA+25] Fuheng Zhao, Divyakant Agrawal, Amr El Abbadi, Claire Mathieu, Ahmed Metwally, and Michel de Rougemont. The spacesaving±family of algorithms for data streams with bounded deletions. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*, pages 3153–3164. IEEE, 2025.